

CALIFORNIA STATE UNIVERSITY NORTHRIDGE

IMPLEMENTATION OF COMPLETE ASIC DESIGN USING LOW POWER
METHODOLOGY

A graduate project submitted in partial fulfillment of the requirements
For the degree of Masters of Science
in Electrical Engineering

By

Mehran Amrbar

December 2012

The graduate project of Mehran Amrbar is approved:

Dr. Ali Amini

Date

Dr. Nagi El Naga

Date

Dr. Ramin Roosta, Chair

Date

California State University, Northridge

Acknowledgement

This project work would not have been possible without the guidance and the help of several individuals who in one way or another contributed and extended their valuable assistance in the preparation and completion of this project.

I would like to show my greatest appreciation to Dr. Ramin Roosta for his tremendous support and help. Dr. Roosta's continuous guidance, support and resourcefulness in the field of ASIC helped me throughout my master program and my project. Without his constant support and help this project would not have been materialized. It was an honor for me to work under his guidance.

I would like to show my gratitude to Dr. Ali Amini for being a mentor and for his continuous guidance during the full course of my studies at CSUN as the Chair of ECE department and for his support as a member of project committee. I would like to thank Dr. Nagi El Naga for his valued attention, information and support as graduate coordinator and as a member of project committee.

Special thanks to Dave Wilder, Synopsys lecturer, for outstanding training on IC Compiler course. I would also like to thank the IT team of ECE department for their support on workstations at EDA/ASIC lab.

Most importantly, I would like to thank my family, friends and especially my wife for their constant support throughout my master program.

Table of Contents

Signature Page.....	ii
Acknowledgement	iii
List of Figures	v
Abstract.....	vi
CHAPTER 1: Understanding VLSI, ASIC and its Flow.....	1
CHAPTER 2: Power Optimizations in ASICs.....	9
CHAPTER 3: Introduction to IC Compiler.....	13
CHAPTER 4: Design Planning.....	24
CHAPTER 5: Placement, Clock Tree Synthesis and Routing Using Zroute.....	34
CHAPTER 6: Analysis of Results and Conclusion.....	49
REFERENCES.....	51
APPENDIX A.....	52
APPENDIX B.....	59

List of Figures

Figure 1.1: Cell Based ASIC (CBIC) Die with One Flexible Block and Few Macros.....	2
Figure 1.2: Channel Gate Array.....	3
Figure 1.3: Channeless Gate Array.....	4
Figure 1.4: Structured Gate Array ASIC.....	5
Figure 1.5: Traditional ASIC Design Flow.....	8
Figure 2.1: Switching Power Dissipation.....	10
Figure 2.2: Components of Power Dissipation.....	10
Figure 2.3: Static Leakage Currents.....	11
Figure 2.4: Latch Based Clock Gating.....	12
Figure 3.1 - IC Compiler Design Flow.....	14
Figure 4.1: IC Compiler Design Planning.....	24
Figure 5.1: Clock Tree Synthesis – Initial Clock.....	36
Figure 5.2: Basic ZRoute Flow.....	40
Figure A.1: RISC CHIP Floorplanned.....	52
Figure A.2: RISC CHIP Placed.....	53
Figure A.3: Initial Routing.....	54
Figure A.4: Power Density.....	55
Figure A.5: Clock Tree.....	56
Figure A.6: Detailed Routing.....	57
Figure A.7: Final Layout.....	58

Abstract

IMPLEMENTATION OF COMPLETE ASIC DESIGN USING LOW POWER METHODOLOGY

By

Mehran Amrbar

Master of Science in Electrical Engineering

In today's technological advancements in VLSI industry, the limits of ASICs/FPGA chips in terms of area, power and speed are constantly shrinking. The end user requirements are also influencing these limits and pushing them to a new level on top of all these technological advancements. The effects of nanometer technologies on congestion, signal integrity, crosstalk etc. are becoming more significant as the technology sizes of semiconductor devices continue to decrease. All of these factors are affecting and forcing various technological methodologies throughout the design flow to constantly fight and keep updating the EDA tools to cop-up with these issues.

Thus, there is always a need of constant learning and exposure to new advanced EDA tools like Synopsys Design Compiler, IC Compiler, PrimeTime, TetraMax etc. The aim of this project is to successfully complete the ASIC design flow with low power techniques, using the advance industry level tools. This project provides a solid base and practical hands-on experience of these advanced tools. It also provides an overview of types of ASICs, detailed ASIC standard design flow and Synopsys IC compiler flow. Along with this, the analysis of various design factors affecting the performance of the final chip such as power, area and timing is also performed.

In this project, a RISC CHIP from Synopsys will be used to perform ASIC design flow and low power methodology.

CHAPTER 1: Introduction to VLSI, ASIC and its Flow

ASICs are application specific integrated circuits, in other words they are integrated circuits designed for a specific purpose and are different from FPGAs, which are used for wide variety of general applications. There are other differences between ASICs and FPGAs as we discuss some of them in this chapter. ASICs are used everywhere in any device from consumer electronics to medical, defense and space industries. [1]

ASICs are different from FPGAs in these terms:

- FPGAs are reprogrammable but ASICs are one-time programmable
- FPGAs have lower initial cost but ASICs have a high initial cost
- FPGAs have higher cost per unit compare to ASICs
- FPGAs have lower NRE (non-recurring engineering) costs upfront compare to ASICs
- ASICs are better in terms of Performance (speed), Area and Power than FPGAs
- FPGAs have a lot more ready IPs than ASICs

ASICs are categorized in these types:

1-Full custom ASICs

Full-custom ASIC's are those that are entirely tailor-fitted to a particular application from the very start. Since its ultimate design and functionality is pre-specified by the user, it is manufactured with all the photolithographic layers of the device already fully defined, just like most off-the-shelf general purpose IC's. These designs are referred to "handcrafted" designs. The use of predefined masks for manufacturing leaves no option for circuit modification during fabrication, except perhaps for some minor fine-tuning or calibration. This means that a full-custom ASIC cannot be modified to suit different applications, and is generally produced as a single, specific product for a particular application only. [1]

2-Semi-Custom ASICs

Semi-custom ASIC's, can be partly customized to serve different functions within its general area of application. Unlike full-custom ASIC's, semi-custom ASIC's are designed to allow a certain degree of modification during the manufacturing process. A semi-custom ASIC is manufactured with the masks for the diffused layers already fully defined, so the transistors and other active components of the circuit are already fixed for that semi-custom ASIC design. The customization of the final ASIC product to the

intended application is done by varying the masks of the interconnection layers, e.g., the metallization layers. [1]

3-Standard Cell-based ASICs

Cell-based ASICs (CBICs) make use of pre-designed, pre-tested, and pre-characterized logic cells such as AND gates, OR gates, multiplexers, and flip-flops for the design. These predefined cells are also referred to as standard cells. The standard-cell areas (also called flexible blocks) in a CBIC are built of rows of standard cells similar to a wall built of bricks. The standard-cell areas may be used in combination with larger pre-designed cells, known as megacells. Megacells are also known as mega-functions, full-custom blocks, system-level macros (SLMs), fixed blocks, cores, or Functional Standard Blocks (FSBs). Examples of such megacells are microcontrollers and microprocessors. Figure 1.1 shows the arrangement of a Cell Based ASIC with one flexible block and few macros. [2]

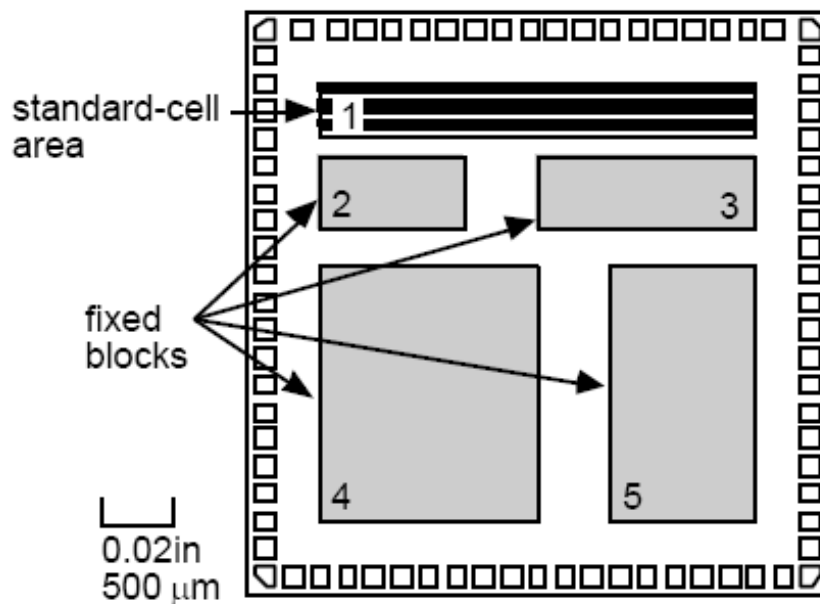


Figure 1.1: A Cell Based ASIC (CBIC) die with one flexible block and few macros [12]

4- Array-Based ASICs

A-Channeled Gate Array

Channeled Gate arrays contain empty channels of silicon separated into rows of unwired transistor pairs, which can be configured into gates, flip-flops or large functions. The routing between the elements is performed by using the dedicated routing channels. Figure 1.2 shows the Channeled Gate Array while Figure 1.3 shows the Channelless Gate Array. In channeled gate arrays, the interconnect is customized. The interconnect uses predefined spaces between rows of base cells. [1]

B-Channelless Gate Array

Channelless architecture is used for designs beyond the limit of channeled because they offer more efficient routing with the sea-of-gates approach. The channeled arrays are reaching three and four layers with increasingly test and integration efforts. In the channelless arrays there are no pre-defined areas set aside for routing between cells. [2]

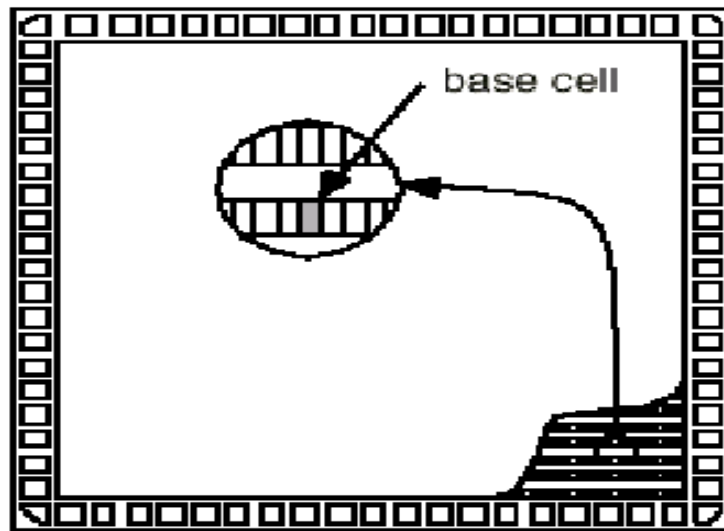


Figure 1.2 Channeled Gate Array[12]

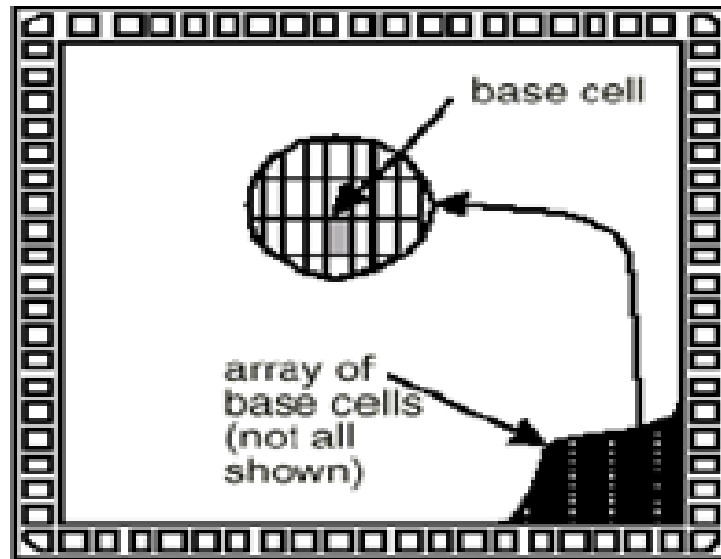


Figure 1.3 Channelless Gate Array[12]

C-Structured or Platform

Structured Gate Arrays combine some of the features of standard Cell based ASICs and MGAs. These are designed and produced from a tightly defined set of: 1) design methodologies; 2) intellectual properties (IP's); 3) well-characterized silicon, aimed at shortening the design cycle and minimizing the development costs of the ASIC. Figure 1.4 demonstrates the Structured Gate Array with existing embedded block. A platform ASIC is built from a group of 'platform slices', with a 'platform slice' being defined as a pre-manufactured device, system, or logic for that platform. Each slice used by the ASIC may be customized by varying its metal layers. The “re-use” of pre-manufactured and pre-characterized platform slices simply means that platform ASIC's are not built from scratch, thereby minimizing design cycle time and costs. [1]

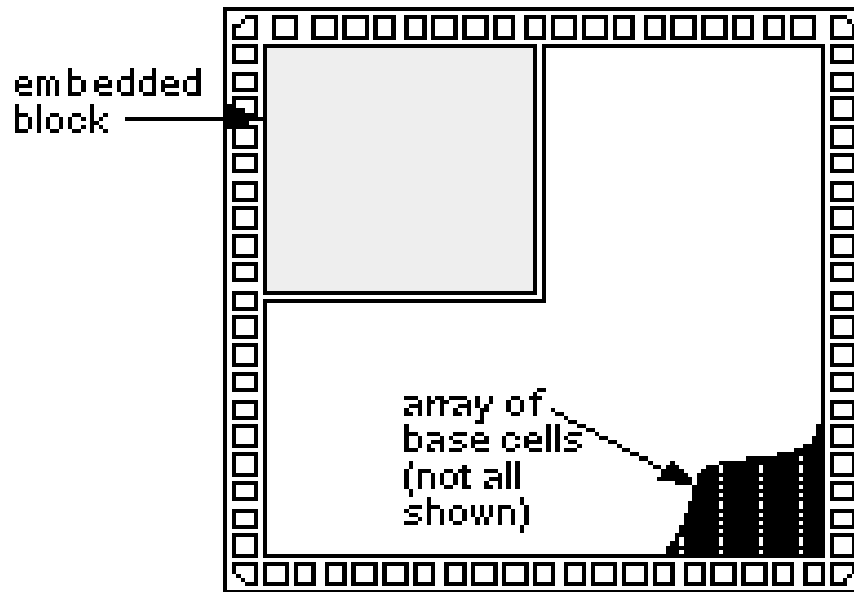


Figure 1.4 Structured Gate Array ASIC[12]

1.1 VLSI flow: Analogy of VLSI and Building Architecture - a thought process

VLSI flow consists of following steps:

Building a Floor-plan: to plan the design of the chip and how the blocks are being partitioned in the area of the chip in order to minimize the area and to minimize the delay between cells.

Power Grid: to design the grids for power and ground in a way that minimizes the power consumption of cells.

Clock-tree: to design the clock tree the way that minimizes the delays and meets the timing constraints. This consists of adding necessary buffers for signal recovery of the clock.

Place and Route: To place the standard cells and macros and route them properly in a way that minimizes area, and meets the timing requirements.

Design for Test (DFT): To insert the scan-chain and make the design testable for future testing of the design. [4]

1.2 ASIC Design Flow

The traditional ASIC design flow contains the steps outlined below:

- i. Prepare requirement specification and create an Micro-Architecture document.
- ii. RTL design and development of IP's.
- iii. After the previous step DFT memory BIST insertion can also be implemented, if the design contains any memory element.
- iv. Functional verification all the IPS. Check whether the RTL is free from linting errors and analyze whether the RTL is synthesis friendly.
 - a. Perform cycle based verification (functional) to verify the protocol behavior of the RTL.
 - b. Perform the property checking to verify the RTL implementation and the specification understanding is matching.
- v. Design environment setting. This includes the technology file to be used along with other environmental attributes.
- vi. Prepare the design constraints file to perform synthesis, usually called as an SDC `synopsys_constraints` or `dc_synopsys_setup` file, specific to synthesis tool (design compiler).

Once the constraints file is set. For performing synthesis inputs to the DC are the library file (for which the synthesis needs to be targeted for, which has the functional/timing information available for the standard cell library and the wire load models for the wires based on the fan-out length of the connectivity), RTL files and the design constraints files, so that the synthesis tool can perform the synthesis of the RTL files and map and optimize to meet the design constraints requirements. After performing the synthesis, scan insertion and JTAG scan chain insertions are implemented and then synthesis is repeated.

- vii. Check whether the design is meeting the requirements after synthesis. Perform block level static timing analysis using Design compiler's built-in static timing analysis engine.
- viii. Perform Formal verification between RTL and the synthesized netlist to confirm that the synthesis tool has not altered the functionality.
- ix. Perform the pre-layout STA (static timing analysis) using PrimeTime with the SDF (standard delay format) file and synthesized netlist file to check whether the design is meeting the timing requirements.
- x. Once the synthesis is performed the synthesized netlist file (VHDL/Verilog format) and the SDC (constraints file) is passed as input files to the Placement and routing tool to perform the back-end activities. The tool used is IC Compiler. [4]
- xi. Initialize the floorplanning with timing driven placement of cells, clock tree insertion and global routing.
- xii. Transfer of clock tree to the original design (netlist) residing in Design Compiler.
- xiii. In-place optimization of the design in Design Compiler.
- xiv. Formal verification between the synthesized netlist and clock tree inserted netlist, using Formality.
- xv. Extraction of estimated timing delays from the layout after the global routing step.
- xvi. Back annotation of estimated timing data from the global routed design, to PrimeTime.
- xvii. Static timing analysis in PrimeTime, using the estimated delays extracted after performing global route.
- xviii. Detailed routing of the design.
- xix. Extraction of real timing delays from the detailed routed design.
- xx. Back annotation of the real extracted timing data to PrimeTime.
- xxi. Post-layout static timing analysis using PrimeTime.
- xxii. Functional gate-level simulation of the design with post-layout timing (if desired).
- xxiii. Tape out after LVS and DRC verification.[5]

The traditional ASIC design flow contains the steps graphically illustrates in the Figure 1.5 below.

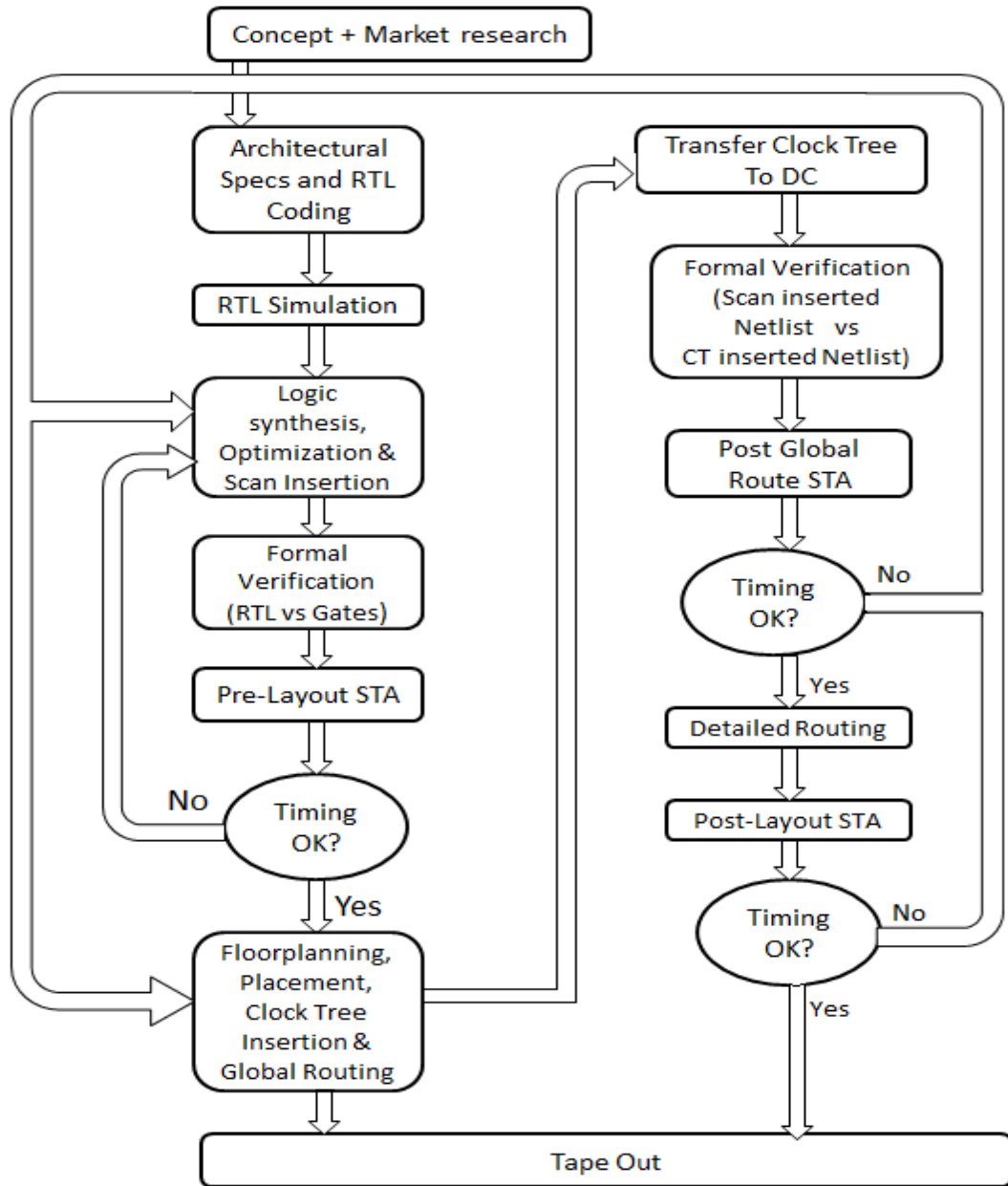


Figure 1.5 Traditional ASIC Design Flow [5]

CHAPTER 2: Power Optimization in ASICs

2.1 Introduction to Power Reduction Techniques

IC Compiler can provide various power reduction techniques, including clock gating, operand isolation, multivoltage leakage power optimization, and gate-level power optimization.

- Leakage power or static power optimization
 - 1) Multivoltage threshold power optimization
 - 2) Power switching
- Dynamic power optimization
 - 1) Clock-gating cell insertion techniques: discrete components, integrated clock gating, generic integrated clock gating
 - 2) Operand isolation
 - 3) Gate-level power optimization
- Multivoltage and Multicorner-Multimode support Power Compiler provides a complete methodology for low-power designs
- Power optimization technology

2.2 Power Modeling

There are two types of power consumptions to consider, dynamic and static. Dynamic power is consumed during the switching of transistors, so it depends on the clock frequency and switching activity.

2.2.1 Dynamic Power

The cause of switching power is illustrated in Figure 2.1. A transition from 0 to 1 on the output of the inverter charges the capacitive load of the output net through the PMOS transistor. A transition from 1 to 0 discharges the same capacitive load through the NMOS. Internal power is any power dissipated within the boundary of a cell. During switching, a circuit dissipates internal power by the charging or discharging of any existing capacitances internal to the cell. Internal power includes power dissipated by a momentary short circuit between the P and N transistors of a gate, called short-circuit power. [13]

To illustrate the cause of short-circuit power, consider the simple gate shown in Figure 2.2. A rising signal is applied at IN. As the signal transitions from low to high, the N type transistor turns on and the P type transistor turns off. However, for a short time during signal transition, both the P and N type transistors can be on simultaneously. During this time, current I_{sc} flows from V_{dd} to GND, causing the dissipation of short-circuit power (P_{sc}). [13]

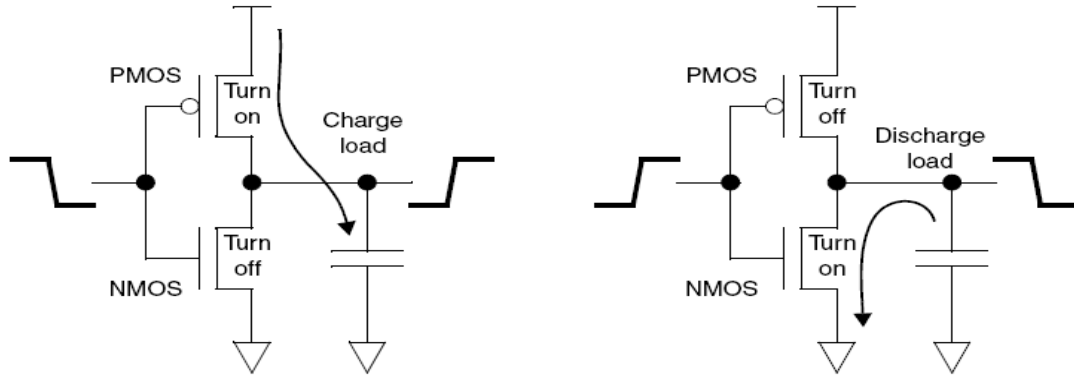


Figure 2.1 - Switching Power Dissipation [13]

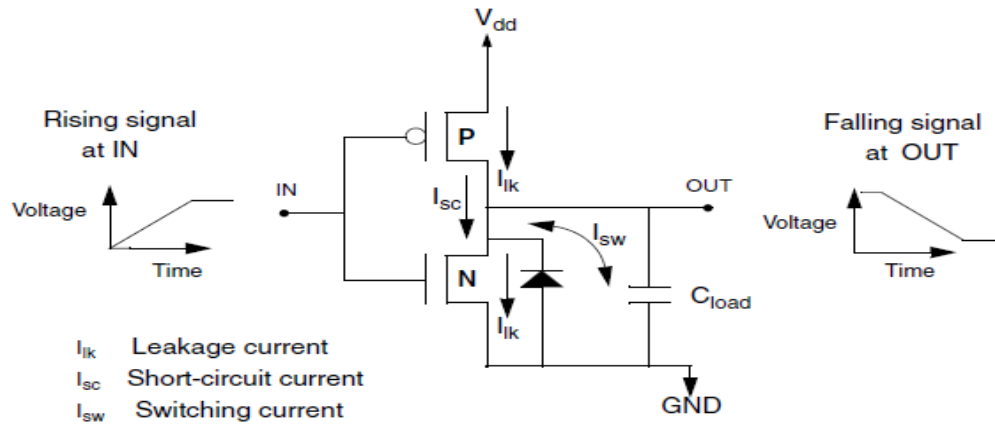


Figure 2.2 - Components of Power Dissipation [14]

2.2.2 Static (Leakage) Power

Static power is the transistor leakage current that flows whenever power is applied to the device, so it is not related to the clock frequency or switching activity. Leakage current was negligible in earlier CMOS technologies. However, with shrinking device geometries and reduced threshold voltages, leakage power is becoming increasingly significant, sometimes approaching the levels of dynamic power dissipation. The main causes of leakage power are reverse-bias p-n junction diode leakage, sub-threshold leakage, and gate leakage. These leakage paths in a CMOS inverter are shown in Figure 2.3. [14]

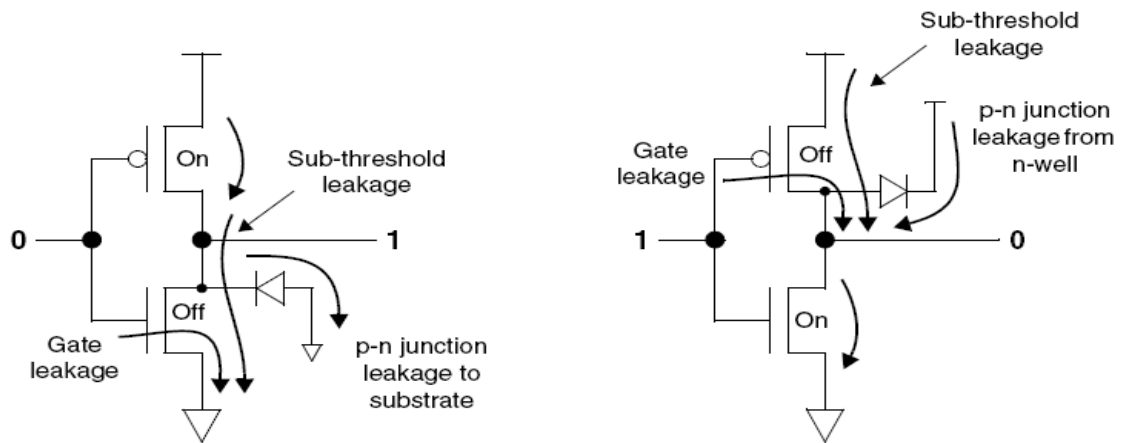


Figure 2.3 - Static Leakage Currents [13]

2.3 Clock Gating

Clock gating applies to synchronous load-enable registers, which are groups of flip-flops that share the same clock and synchronous control signals and that are inferred from the same HDL variable. Synchronous control signals include synchronous load-enable, synchronous set, synchronous reset, and synchronous toggle. The registers are implemented by Design Compiler by use of feedback loops. However, these registers maintain the same logic value through multiple cycles and unnecessarily use power. Clock gating saves power by eliminating the unnecessary activity associated with reloading register banks. Designs that benefit most from clock gating are those with low-throughput datapaths. Designs that benefit less from RTL clock gating include designs

with finite state machines or designs with throughput-of-one datapaths. Figure 2.4 shows a latch-based clock-gating style using a 2-input AND gate; however, depending on the type of register and the gating style, gating can use NAND, OR, and NOR gates instead.

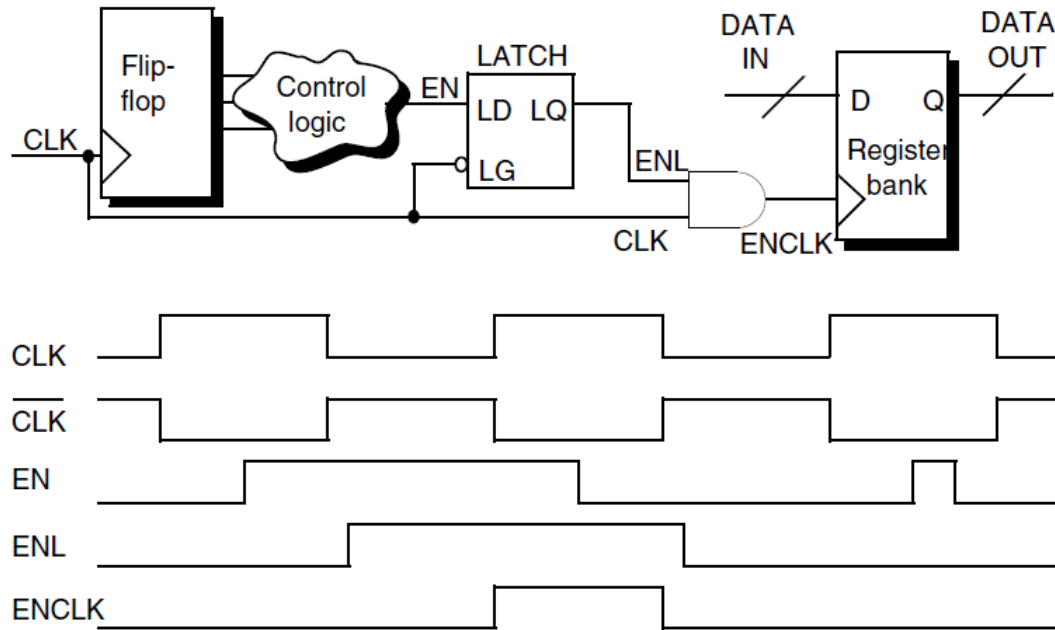


Figure 2.4 - Latch Based Clock Gating [14]

CHAPTER 3: Introduction to IC Compiler

3.1 Introduction

IC Compiler is the software package from Synopsys for Physical Design of ASIC. It provides necessary tools to complete the back end design of the very deep submicron designs.

The inputs to the IC Compiler are: a gate-level netlist which can be from DC Compiler or third-party tools, a detailed floorplan which can be from previous Design Planning through IC Compiler or other third-party tools, timing constraints and other constraints, physical and timing libraries provided by manufacturer, and foundry-process data.

IC Compiler generates a GDSII-format file as output ready for tape out of the chip. In addition, it is possible to export a Design Exchange Format (DEF) file of placed netlist data ready for a third-party router.

IC Compiler uses a binary Synopsys Milkyway database, which can be used by other Synopsys tools based on Milkyway. [16]

The IC Compiler design flow is an easy-to-use, single-pass flow that provides convergent timing closure. Figure 3.1 shows the basic IC Compiler design flow.

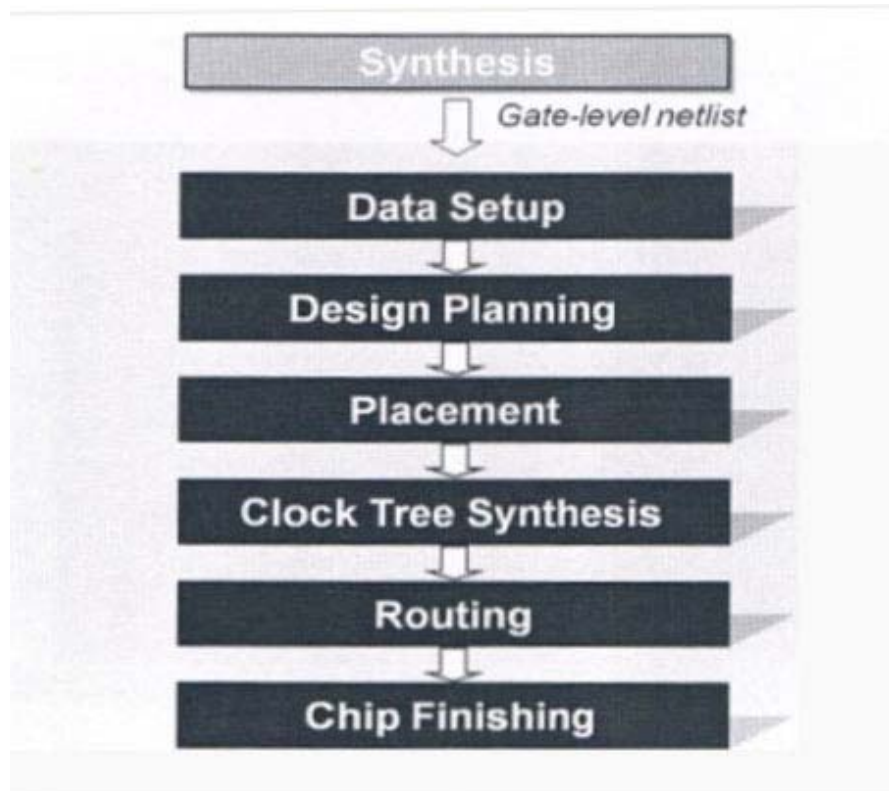


Figure 3.1 - IC Compiler Design Flow [21]

3.2 Data Setup

In this section, we go through the systematic process of the setup of our design and Milkyway database.

1-First, you have to setup the .synopsys_dc.setup file. You can use a UNIX text editor. Here is the setup file with added user-defined variables:

```
# -----  
# Suppress known and/or annoying messages  
# -----  
suppress_message {PSYN-040 PSYN-088 PSYN-058 PSYN-039 PSYN-024 RCEX-060  
PSYN-087 PSYN-850 TFCHK-055}  
  
# Suppress warnings about metal layer pitch that occurs during create_mw_lib:  
suppress_message {TFCHK-049 TFCHK-050}  
  
# Suppress warning that "43 logical cells do not have P/G pins" from check_mv_design -  
power_nets:  
suppress_message {MV-597}  
  
# Suppress warning about ignored DEF syntax and "Information" about "preferred wire  
track direction  
# not being set" during read_def:  
suppress_message {DDEFR-054 MWDEFR-159}  
  
# Suppress warning about "skipping AHFS on don't touch high-fanout nets" during  
place_opt:  
suppress_message {PSYN-1002}  
  
# Suppress warnings about "Ignore pin on layer 0", "Ignore top cell pins with no ports",  
# and "METAL pitch too small" during route_zrt_global -congestion_map_only true:  
suppress_message {ZRT-026 ZRT-027 ZRT-030}  
  
# Suppress warning about "P/G ports being on non-routing layer "UNKNOWN" " and  
warning about  
# using Elmore instead of "clock_arnoldi" delay calculation model during clock_opt:  
suppress_message {MWLIBP-311 CTS-352}  
  
# Suppress warning about "not enough nets being routed" during route_opt:  
suppress_message {RCEX-047}
```

```
# Suppress warning: Power connection/checking is skipped for 2666 power pins because
the required power pin information cannot be found in logical libraries.
suppress_message {MV-510}
```

```
# -----
# Load useful "functions" or procedures, like "view"
# -----
source ../ref/tools/procs.tcl
```

```
# -----
# General useful settings
# -----
# Disable more-like page mode
set_app_var enable_page_mode false
# Don't want to see CMD-041 when creating new variables
set_app_var sh_new_variable_message false
# Increase history buffer from 20 commands to 100
history keep 100
```

```
# -----
# Enable logging of commands and everything by date/shell
# -----
set timestamp [clock format [clock scan now] -format "%Y-%m-%d_%H-%M"]
set sh_output_log_file "${synopsys_program_name}.log.[pid].$timestamp"
set sh_command_log_file "${synopsys_program_name}.cmd.[pid].$timestamp"
```

```
# -----
# Aliases
# -----
alias v view
alias rt "report_timing -nosplit"
alias rtm "report_timing -nosplit -delay min"
alias rc "report_constraint -all_violators -nosplit"
alias rq report_qor
alias h history
alias _ measure_time
```

```

# -----
# Logic Library settings
# -----
lappend search_path ../ref/db ../ref/tlup
set_app_var target_library "sc_max.db"
set_app_var link_library "* sc_max.db io_max.db special_max.db ram16x128_max.db
ram4x32_max.db ram8x64_max.db ram32x64_max.db"
set_min_library sc_max.db -min_version sc_min.db
set_min_library io_max.db -min_version io_min.db
set_min_library special_max.db -min_version special_min.db
set_min_library ram16x128_max.db -min_version ram16x128_min.db
set_min_library ram4x32_max.db -min_version ram4x32_min.db
set_min_library ram8x64_max.db -min_version ram8x64_min.db
set_min_library ram32x64_max.db -min_version ram32x64_min.db

# -----
# RISC_CHIP setup variables
# -----
set my_mw_lib risc_chip.mw
set mw_path "../ref/mw_lib"
set tech_file "../ref/tech/cb13_6m.tf"
set tlup_map "../ref/tlup/cb13_6m.map"
set tlup_max "../ref/tlup/cb13_6m_max.tluplus"
set tlup_min "../ref/tlup/cb13_6m_min.tluplus"
set top_design "RISC_CHIP"
set verilog_file "../design_data/RISC_CHIP.v"
set sdc_file "../design_data/RISC_CHIP.sdc"
set def_file "../design_data/RISC_CHIP.def"
set ctrl_file "../scripts/opt_ctrl.tcl"
set derive_pg_file "../scripts/derive_pg.tcl"
set libs {sc io special ram4x32 ram8x64 ram32x64 ram16x128}
set mw_ref_libs ""
foreach lib $libs {
    lappend mw_ref_libs $mw_path/$lib
}

```

2-Then you invoke the IC Compiler from UNIX prompt:

```
UNIX% icc_shell
```

3-You start the GUI using the command:

```
start_gui
```

4-Then you will create the design library:

Use the MainWindow menu File -> Create Library... to bring up the Create Library dialog box.

Use the variables already defined for the library and tech file names.

Attach reference libraries to your design library:

```
sc, io and ram16x128
```

Select the “Open library” and then click OK.

Or you can use the command:

```
create_mw_lib -technology $tech_file -mw_reference_library  
{/home/users5/ma581955/ref/mw_lib/sc /home/users5/ma581955/ref/mw_lib/io  
/home/users5/ma581955/ref/mw_lib/ram16x128} -bus_naming_style {[%d]} -open  
$my_mw_lib
```

5-Then you have to import design files:

```
import_designs $verilog_file -format verilog -top %top_design
```

6-Then you setup the technology files:

```
set_tlu_plus_files -max_tluplus /home/users5/ma581955/ref/tlup/cb13_6m_max.tluplus  
-min_tluplus /home/users5/ma581955/ref/tlup/cb13_6m_min.tluplus -tech2itf_map  
/home/users5/ma581955/ref/tlup/cb13_6m.map
```

7-Then you define the “logical” connections between power/ground pins and nets:

```
source $derive_pg_file  
#@ derive_pg_connection -power_net VDD -power_pin VDD -ground_net VSS -  
ground_pin VSS  
#@ derive_pg_connection -power_net VDDO -power_pin VDDO -ground_net VSSO -  
ground_pin VSSO
```



```

#@ derive_pg_connection -power_net VDDQ -power_pin VDDQ -ground_net VSSQ -
ground_pin VSSQ
#@ derive_pg_connection -power_net VDD -ground_net VSS -tie
#@ # -- End

```

8-There, should be no “unconnected” power or ground pins, check with this command:

```
check_mv_design -power_nets
```

9-Apply the top-level design constraints:

```

read_sdc $sdc_file
#@ define_proc_attributes all_physical_only_cells -info "Get the collection of physical
only cells" -define_args {
#@ { -coordinates "coordinates of bounding box" "llx lly urx ury" string {optional}}
#@ { -lib_cells "get cells related to these objects" "objects" list {optional}}
#@ { -cell_name "list of cell name patterns and collections" "patterns" list {optional}}
#@ { "" "Existing collection" "arguments" list {optional}}
#@ }
#@ define_proc_attributes all_physical_only_nets -info "Get the collection of physical
only nets" -define_args {
#@ { "" "list of cell name patterns and collections" "arguments" list {optional}}
#@ }
#@ define_proc_attributes all_drc_violated_nets -info "Returns all DRC violated nets" -
define_args {
#@ { -max_capacitance "max Cap DRC violated nets" "" string {optional}}
#@ { -max_transition "max transition time DRC violated nets" "" string {optional}}
#@ { -max_fanout "max fanout DRC violated nets" "" string {optional}}
#@ { -threshold "Captures all values more than or equal to threshold" "threshold" string
{optional}}
#@ { -bound "Captures all values less than or equal to bound bound" "upper" string
{optional}}
#@ { "" "objects patterns OR collection" "arguments" list {optional}}
#@ }
#@ define_proc_attributes all_fanin -info "return pins or cells in transitive fanin" -
define_args {
#@ { -to "set of sink ports, pins, or nets" "sink_list" list {optional}}
#@ { -startpoints_only "return only the timing startpoints" "" string {optional}}
#@ { -exclude_bboxes "black-boxes are not included in return-set" "" string {optional}}
#@ { -break_on_bboxes "traversal stops at black-boxes" "" string {optional}}
#@ { -only_cells "only cells are included in the return-set" "" string {optional}}

```

```

#@ { -flat "trace entire design instead of just current hierarchy" "" string {optional}}
#@ { -levels "maximum number of levels of logic to traverse" "n" string {optional}}
#@ }
#@ define_proc_attributes all_high_fanout -info "Returns all high fanout objects" -
define_args {
#@ { -nets "high fanout nets" "" string {optional}}
#@ { -threshold "high fanout threshold: value>=0" "threshold" string {optional}}
#@ { -through_buf_inv "treat buffers and inverters as transparent -check for transitive
fanout" "" string {optional}}
#@ { "" "objects patterns OR collection" "arguments" string {optional}}
#@ }
#@ define_proc_attributes all_ideal_nets -info "Returns all ideal nets" -define_args {
#@ { "" "objects patterns OR collection" "arguments" string {optional}}
#@ }
#@ define_proc_attributes all_inputs -info "return input ports" -define_args {
#@ { -clock "ports with input delay relative to clock" "clock" string {optional}}
#@ { -edge_triggered "ports with non-level-sensitive input delay" "" string {optional}}
#@ { -level_sensitive "ports with level-sensitive input delay" "" string {optional}}
#@ }
#@ define_proc_attributes all_outputs -info "return output ports" -define_args {
#@ { -clock "ports with output delay relative to clock" "clock" string {optional}}
#@ { -edge_triggered "ports with non-level-sensitive output delay" "" string {optional}}
#@ { -level_sensitive "ports with level-sensitive output delay" "" string {optional}}
#@ }
#@ define_proc_attributes all_threestate -info "Returns all threestate objects" -
define_args {
#@ { -nets "threestate nets" "" string {optional}}
#@ { "" "objects patterns OR collection" "arguments" string {optional}}
#@ }
#@ define_proc_attributes all_dont_touch -info "Returns all dont touch objects" -
define_args {
#@ { -nets "Dont touch nets" "" string {optional}}
#@ { -cells "Dont touch cells" "" string {optional}}
#@ { "" "Dont touch patterns OR collections" "patterns" string {optional}}
#@ }
#@ define_proc_attributes all_size_only_cells -info "Returns all size_only cells" -
define_args {
#@ { "" "Object patterns OD collections" "patterns" string {optional}}
#@ }
#@ define_proc_attributes all_clocks -info "return all clocks in the current design"
#@ define_proc_attributes all_designs -info "return all designs"

```

```

#@ define_proc_attributes all_registers -info "create a collection of register cells or
pins" -define_args {
#@ { -no_hierarchy "only search the top level of hierarchy" "" string {optional}}
#@ { -rise_clock "return registers triggered by rise edge of this clock" "rise_clock" string
{optional}}
#@ { -fall_clock "return registers triggered by rise edge of this clock" "fall_clock" string
{optional}}
#@ { -clock "return registers of this clock" "clock" string {optional}}
#@ { -cells "return cells - this is the default" "" string {optional}}
#@ { -data_pins "return data_pins" "" string {optional}}
#@ { -clock_pins "return clock_pins" "" string {optional}}
#@ { -slave_clock_pins "return slave_clock_pin" "" string {optional}}
#@ { -inverted_output "return FF's with inverted output phase" "" string {optional}}
#@ { -output_pins "return output_pins" "" string {optional}}
#@ { -level_sensitive "only consider level sensitive registers" "" string {optional}}
#@ { -edge_triggered "only consider edge triggered register" "" string {optional}}
#@ { -master_slave "only consider master/slave registers" "" string {optional}}
#@ }
#@ define_proc_attributes all_critical_cells -info "return critical cells" -define_args {
#@ { -slack_range "critical range to select cells:value>=0" "range_value" string
{optional}}
#@ }
#@ define_proc_attributes all_critical_pins -info "return critical pins" -define_args {
#@ { -slack_range "critical range to select cells:value>=0" "range_value" string
{optional}}
#@ { -type "pin_type: Values: endpoint, startpoint" "pin_type" string {optional}}
#@ }
#@ define_proc_attributes get_magnet_cells -info "return magnet cells collection" -
define_args {
#@ { -stop_by_sequential_cells "don't pull sequential cells" "" string {optional}}
#@ { -exclude_buffers "skip through buffers in levels" "" string {optional}}
#@ { -logical_level "number of logical levels to pull for placement" "integer" string
{optional}}
#@ { -stop_points "list of stop points where to stop magnetizing paths" "object_list" list
{optional}}
#@ { "" "a list of magnet objects" "magnet_objects" list {optional}}
#@ }
#@ define_proc_attributes all_fanout -info "return pins or cells in transitive fanout" -
define_args {
#@ { -clock_tree "return all clock tree components in design" "" string {optional}}
#@ { -from "set of source ports, pins, or nets" "source_list" list {optional}}

```

```

#@ {-endpoints_only "return only the timing endpoints" "" string {optional}}
#@ {-exclude_bboxes "black-boxes are not included in return-set" "" string {optional}}
#@ {-break_on_bboxes "traversal stops at black-boxes" "" string {optional}}
#@ {-only_cells "only cells are included in the return-set" "" string {optional}}
#@ {-flat "trace entire design instead of just current hierarchy" "" string {optional}}
#@ {-levels "maximum number of levels of logic to traverse" "n" string {optional}}
#@ }
#@ define_proc_attributes all_physical_only_ports -info " Get the collection of physical
only ports" -define_args {
#@ {"" "list of cell name patterns and collections" "arguments" list {optional}}
#@ }
#@ # -- End source /usr/synopsys/E-2010.12-SP2-
ICC/auxx/syn/all_command_option_def.tcl

```

10-Then apply some timing and optimization controls which are specified in opt_ctrl.tcl

```

source $ctrl_file
#@ # -----
#@ # General timing and optimization control settings
#@ # -----
#@ set_app_var timing_enable_multiple_clocks_per_reg true
#@ set_app_var case_analysis_with_logic_constants true
#@ set_app_var physopt_delete_unloaded_cells false
#@ set_app_var physopt_power_critical_range 0.8
#@ set_app_var physopt_area_critical_range 0.8
#@ set_app_var enable_recovery_removal_arcs true
#@
#@ # set_false_path from <clock_name> -to <clock_name>
#@ set_fix_multiple_port_nets -all -buffer_constants
#@ set_auto_disable_drc_nets -constant false
#@ set_timing_derate -max -early 0.95
#@ # set_dont_use <off_limit_cells>
#@ # set_prefer -min <hold_fixing_cells>
#@ set_max_area 0
#@ group_path -name INPUTS -from [all_inputs]
#@ group_path -name OUTPUTS -to [all_outputs]
#@ group_path -name COMBO -from [all_inputs] -to [all_outputs]
#@ set_ideal_network [all_fanout -flat -clock_tree]
#@ # set_cost_priority {max_transistion max_delay}
#@ # -- End source ./scripts/opt_ctrl.tcl

```

11-The “scan enable” signal is defined as an ideal network to prevent synthesis from buffering this signal. Remove the ideal network to make it possible for tool to buffer the signal during physical design:

```
remove_ideal_network [get_ports scan_en]
```

CHAPTER 4: Design Planning

4.1 Introduction

Design Planning is the iterative process of creating a floorplan.

A chip-level floorplan entail defining:

- core size, shape and placement rows
- periphery: IO, power, corner and filler pad cell locations
- macro cell placement
- standard cell placement constraints (blockages)
- power grid (rings, straps, rails)

Figure 4.1 shows IC compiler Design Planning. It shows how chip-level floorplan is different from block-level floorplan. [21]

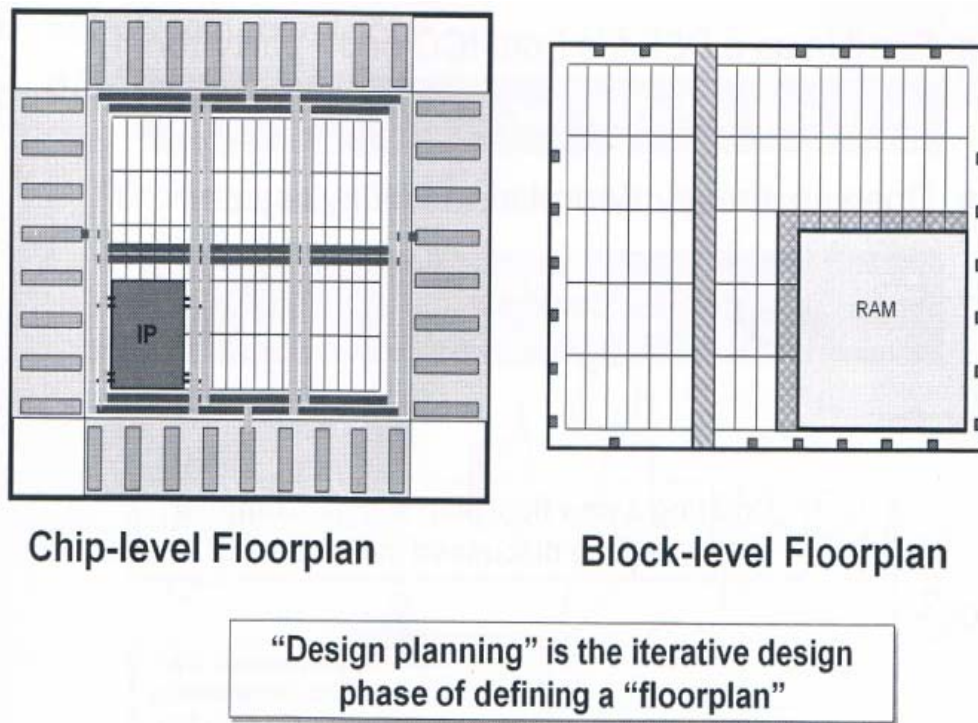


Figure 4.1 - IC Compiler Design Planning [21]

A physical design or layout is the result of a synthesized netlist that has been placed and routed.

4.2 Procedure

1-Open design library and design cell:

2-Apply timing and optimization controls:

```
source scripts/opt_ctrl.tcl
```

```
##@ # -----
##@ # General timing and optimization control settings
##@ # -----
##@ set_app_var timing_enable_multiple_clocks_per_reg true
##@ set_app_var case_analysis_with_logic_constants true
##@ set_app_var physopt_delete_unloaded_cells false
##@ # set physopt_power_critical_range 0.8
##@ # set physopt_area_critical_range 0.8
##@ set_app_var enable_recovery_removal_arcs true
##@
##@ # set_false_path from <clock_name> -to <clock_name>
##@ set_fix_multiple_port_nets -all -buffer_constants
##@ set_auto_disable_drc_nets -constant false
##@ set_timing_derate -max -early 0.95
##@ # set_dont_use <off_limit_cells>
##@ # set_prefer -min <hold_fixing_cells>
##@ set_max_area 0
##@ group_path -name INPUTS -from [all_inputs]
##@ group_path -name OUTPUTS -to [all_outputs]
##@ group_path -name COMBO -from [all_inputs] -to [all_outputs]
##@ set_ideal_network [all_fanout -flat -clock_tree]
##@ # set_cost_priority {max_transistion max_delay}
##@ # -- End source
```

3-In Layout Window, switch task menu to Design Planning:

Select File -> Task -> Design Planning

4-Create the corner P/G cells and define all pad cell positions:

```
source -echo scripts/pad_cell_cons.tcl
```

```
##@ # Create corners and P/G pads
```

```

#@ create_cell {cornerll cornerlr cornerul cornerur} pfrelr
#@ create_cell {vss1left vss1right vss1top vss1bottom} pv0i
#@ create_cell {vdd1left vdd1right vdd1top vdd1bottom} pvdi
#@ create_cell {vss2left vss2right vss2top vss2bottom} pv0a
#@ create_cell {vdd2left vdd2right vdd2top vdd2bottom} pvda
#@
#@ # Define corner pad locations
#@ set_pad_physical_constraints -pad_name "cornerul" -side 1
#@ set_pad_physical_constraints -pad_name "cornerur" -side 2
#@ set_pad_physical_constraints -pad_name "cornerlr" -side 3
#@ set_pad_physical_constraints -pad_name "cornerll" -side 4
#@
#@ # Define signal and PG pad locations
#@
#@ # Left side
#@ set_pad_physical_constraints -pad_name "pad_iopad_0" -side 1 -order 1
#@ set_pad_physical_constraints -pad_name "pad_iopad_1" -side 1 -order 2
#@ set_pad_physical_constraints -pad_name "pad_iopad_2" -side 1 -order 3
#@ set_pad_physical_constraints -pad_name "pad_iopad_3" -side 1 -order 4
#@ set_pad_physical_constraints -pad_name "pad_iopad_4" -side 1 -order 5
#@ set_pad_physical_constraints -pad_name "pad_iopad_5" -side 1 -order 6
#@ set_pad_physical_constraints -pad_name "pad_iopad_6" -side 1 -order 7
#@ set_pad_physical_constraints -pad_name "pad_iopad_7" -side 1 -order 8
#@ set_pad_physical_constraints -pad_name "pad_iopad_8" -side 1 -order 9
#@ set_pad_physical_constraints -pad_name "vdd2left" -side 1 -order 10
#@ set_pad_physical_constraints -pad_name "vdd1left" -side 1 -order 11
#@ set_pad_physical_constraints -pad_name "vss1left" -side 1 -order 12
#@ set_pad_physical_constraints -pad_name "vss2left" -side 1 -order 13
#@ set_pad_physical_constraints -pad_name "pad_iopad_9" -side 1 -order 14
#@ set_pad_physical_constraints -pad_name "pad_iopad_10" -side 1 -order 15
#@ set_pad_physical_constraints -pad_name "pad_iopad_11" -side 1 -order 16
#@ set_pad_physical_constraints -pad_name "pad_iopad_12" -side 1 -order 17
#@ set_pad_physical_constraints -pad_name "pad_iopad_13" -side 1 -order 18
#@ set_pad_physical_constraints -pad_name "pad_iopad_14" -side 1 -order 19
#@ set_pad_physical_constraints -pad_name "pad_iopad_15" -side 1 -order 20
#@ set_pad_physical_constraints -pad_name "pclk_iopad" -side 1 -order 21
#@ set_pad_physical_constraints -pad_name "pll_bypass_iopad" -side 1 -order 22
#@
#@ # Top side
#@ set_pad_physical_constraints -pad_name "pc_be_iopad_0" -side 2 -order 1
#@ set_pad_physical_constraints -pad_name "pc_be_iopad_1" -side 2 -order 2

```



```

#@ set_pad_physical_constraints -pad_name "pc_be_iopad_2" -side 2 -order 3
#@ set_pad_physical_constraints -pad_name "pc_be_iopad_3" -side 2 -order 4
#@ set_pad_physical_constraints -pad_name "pdevsel_n_iopad" -side 2 -order 5
#@ set_pad_physical_constraints -pad_name "pframe_n_iopad" -side 2 -order 6
#@ set_pad_physical_constraints -pad_name "pgnt_n_iopad" -side 2 -order 7
#@ set_pad_physical_constraints -pad_name "pidsel_iopad" -side 2 -order 8
#@ set_pad_physical_constraints -pad_name "pirdy_n_iopad" -side 2 -order 9
#@ set_pad_physical_constraints -pad_name "vdd2top" -side 2 -order 10
#@ set_pad_physical_constraints -pad_name "vdd1top" -side 2 -order 11
#@ set_pad_physical_constraints -pad_name "vss1top" -side 2 -order 12
#@ set_pad_physical_constraints -pad_name "vss2top" -side 2 -order 13
#@ set_pad_physical_constraints -pad_name "ppar_iopad" -side 2 -order 14
#@ set_pad_physical_constraints -pad_name "pperr_n_iopad" -side 2 -order 15
#@ set_pad_physical_constraints -pad_name "preq_n_iopad" -side 2 -order 16
#@ set_pad_physical_constraints -pad_name "prst_n_iopad" -side 2 -order 17
#@ set_pad_physical_constraints -pad_name "pstop_n_iopad" -side 2 -order 18
#@ set_pad_physical_constraints -pad_name "pserr_n_iopad" -side 2 -order 19
#@ set_pad_physical_constraints -pad_name "ptrdy_n_iopad" -side 2 -order 20
#@ set_pad_physical_constraints -pad_name "sys_clk_iopad" -side 2 -order 21
#@ set_pad_physical_constraints -pad_name "sdr_clk_iopad" -side 2 -order 22
#@
#@ # Right side
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_0" -side 3 -order 1
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_1" -side 3 -order 2
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_2" -side 3 -order 3
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_3" -side 3 -order 4
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_4" -side 3 -order 5
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_5" -side 3 -order 6
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_6" -side 3 -order 7
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_7" -side 3 -order 8
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_8" -side 3 -order 9
#@ set_pad_physical_constraints -pad_name "vdd2right" -side 3 -order 10
#@ set_pad_physical_constraints -pad_name "vdd1right" -side 3 -order 11
#@ set_pad_physical_constraints -pad_name "vss1right" -side 3 -order 12
#@ set_pad_physical_constraints -pad_name "vss2right" -side 3 -order 13
#@ set_pad_physical_constraints -pad_name "sdram_A_iopad_9" -side 3 -order 14
#@ set_pad_physical_constraints -pad_name "sdram_BWS_iopad_0" -side 3 -order 15
#@ set_pad_physical_constraints -pad_name "sdram_BWS_iopad_1" -side 3 -order 16
#@ set_pad_physical_constraints -pad_name "sdram_LD_iopad" -side 3 -order 17
#@ set_pad_physical_constraints -pad_name "sdram_RW_iopad" -side 3 -order 18
#@ set_pad_physical_constraints -pad_name "powersave_iopad" -side 3 -order 19

```

```

#@ set_pad_physical_constraints -pad_name "scan_en_iopad" -side 3 -order 20
#@ set_pad_physical_constraints -pad_name "pm66en_iopad" -side 3 -order 21
#@ set_pad_physical_constraints -pad_name "test_mode_iopad" -side 3 -order 22
#@
#@ # Bottom side
#@ set_pad_physical_constraints -pad_name "sdram_CK_iopad" -side 4 -order 1
#@ set_pad_physical_constraints -pad_name "sdram_CKn_iopad" -side 4 -order 2
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_0" -side 4 -order 3
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_1" -side 4 -order 4
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_2" -side 4 -order 5
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_3" -side 4 -order 6
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_4" -side 4 -order 7
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_5" -side 4 -order 8
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_6" -side 4 -order 9
#@ set_pad_physical_constraints -pad_name "vdd2bottom" -side 4 -order 10
#@ set_pad_physical_constraints -pad_name "vdd1bottom" -side 4 -order 11
#@ set_pad_physical_constraints -pad_name "vss1bottom" -side 4 -order 12
#@ set_pad_physical_constraints -pad_name "vss2bottom" -side 4 -order 13
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_7" -side 4 -order 14
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_8" -side 4 -order 15
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_9" -side 4 -order 16
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_10" -side 4 -order 17
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_11" -side 4 -order 18
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_12" -side 4 -order 19
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_13" -side 4 -order 20
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_14" -side 4 -order 21
#@ set_pad_physical_constraints -pad_name "sdram_DQ_iopad_15" -side 4 -order 22
#@ # -- End source ../lab2_dp/scripts/pad_cell_cons.tcl

```

5-Initialize the floorplan:

Select Floorplan -> Initialize Floorplan...

Change the *Core utilization* to 0.8 (80%)

Change the *Core to left/right/bottom/top* spacing to 30

Click OK

```

initialize_floorplan -core_utilization 0.8 -left_io2core 30 -bottom_io2core 30 -
right_io2core 30 -top_io2core 30

```

6-Press F to fit whole design into view area.

7-Insert the pad fillers to fill the gaps between the pads:

```
source scripts/insert_pad_filler.tcl
```

```
#@ insert_pad_filler -cell "pfeed10000 pfeed05000 pfeed02000 pfeed01000 pfeed00500  
pfeed00200 pfeed00100 pfeed00050 pfeed00010 pfeed00005"  
#@  
#@ # -- End source
```

8-Make the “logical” connection (no physical routing) between power/ground signals and all power/ground I/O pads, macros and standard cells:

```
source -echo scripts/connect_pg.tcl
```

```
#@ derive_pg_connection -power_net VDD -power_pin VDD -ground_net VSS -  
ground_pin VSS  
#@ derive_pg_connection -power_net VDDO -power_pin VDDO -ground_net VSSO -  
ground_pin VSSO  
#@ derive_pg_connection -power_net VDDQ -power_pin VDDQ -ground_net VSSQ -  
ground_pin VSSQ  
#@ derive_pg_connection -power_net VDD -ground_net VSS -tie  
#@ # -- End source
```

9-Build the PAD area power supply rings:

```
create_pad_rings
```

10-You can preplace some macros before the virtual flat placement, in this example we preplace only 3 of them using this script:

```
source -echo scripts/preplace_macros.tcl
```

```
# Set pre-defined placement for the PLLs and CLKMUL macros
```

```
set_attribute [all_macro_cells] is_fixed false  
remove_placement -object_type all
```

```
set obj [get_cells {"I_CLOCK_GEN/I_CLKMUL"} -all]  
set_attribute -quiet $obj orientation FN
```

```

set_attribute -quiet $obj origin {1568.995 1677.765}
set_attribute -quiet $obj is_placed true
set_attribute -quiet $obj is_fixed true
set_attribute -quiet $obj is_soft_fixed false
set_attribute -quiet $obj eco_status eco_reset

set obj [get_cells {"I_CLOCK_GEN/I_PLL_PCI"} -all]
set_attribute -quiet $obj orientation N
set_attribute -quiet $obj origin {375.565 1677.765}
set_attribute -quiet $obj is_placed true
set_attribute -quiet $obj is_fixed true
set_attribute -quiet $obj is_soft_fixed false
set_attribute -quiet $obj eco_status eco_reset

set obj [get_cells {"I_CLOCK_GEN/I_PLL_SD"} -all]
set_attribute -quiet $obj orientation S
set_attribute -quiet $obj origin {1778.995 1777.765}
set_attribute -quiet $obj is_placed true
set_attribute -quiet $obj is_fixed true
set_attribute -quiet $obj is_soft_fixed false
set_attribute -quiet $obj eco_status eco_reset

```

11-Before performing virtual flat placement set a constraint to prevent standard cells from being placed in narrow channels between macros(<10um):

```
set_fp_placement_strategy -sliver_size 10
```

12-Limit the legal placement orientation of some RAMs:

```
source -echo scripts/macro_place_cons.tcl
```

```

#@ ### Apply macro placement constraints:
#@
#@ set_fp_placement_strategy      -auto_grouping high      -macros_on_edge on      -
sliver_size 10  -virtual_IPO on
#@
#@      set_fp_macro_options      -legal_orientation      {W      E}      [get_cells
I_ORCA_TOP/I_PCI_TOP/I_PCI_WRITE_FIFO/PCI_FIFO_RAM_*]
#@ # -- End source

```

13- Set a hard keepout margin of 10 microns around all macros:

```
source -echo scripts/keepout.tcl
```

```
## Set hard keepout margin as 10 for all macros
set_keepout_margin -type hard -all_macros -outer {10 10 10 10}
# -- End source
```

14- Perform a timing driven non-hierarchical VF placement:

```
create_fp_placement -timing_driven -no_hierarchy_gravity
```

15- Lock down all macros:

```
set_dont_touch_placement [all_macro_cells]
```

16- Create P/G rings around macros:

```
source ./scripts/macro_pg_rings.tcl
```

```
##### Build rings around six groups of macros:
##
set_fp_rail_region_constraints -polygon {{366 1304} {366 1680} {763 1680} {763 1304}}
create_fp_group_block_ring -nets {VDD VSS} -horizontal_ring_layer METAL5 -
horizontal_ring_offset 0.78 -horizontal_ring_width 3 -vertical_ring_layer METAL4 -
vertical_ring_offset 0.73 -vertical_ring_width 3 -horizontal_strap_layer METAL5 -
horizontal_strap_width 3 -vertical_strap_layer METAL4 -vertical_strap_width 3
commit_fp_group_block_ring
##
set_fp_rail_region_constraints -polygon {{366 940} {366 1300} {625 1300} {625 940}}
create_fp_group_block_ring -nets {VDD VSS} -horizontal_ring_layer METAL5 -
horizontal_ring_offset 0.78 -horizontal_ring_width 3 -vertical_ring_layer METAL4 -
vertical_ring_offset 0.73 -vertical_ring_width 3 -horizontal_strap_layer METAL5 -
horizontal_strap_width 3 -vertical_strap_layer METAL4 -vertical_strap_width 3
commit_fp_group_block_ring
##
set_fp_rail_region_constraints -polygon {{926 1193} {926 1807} {1454 1807} {1454 1193}}
```

```

#@ create_fp_group_block_ring -nets {VDD VSS} -horizontal_ring_layer METAL5 -
horizontal_ring_offset 0.78 -horizontal_ring_width 3 -vertical_ring_layer METAL4 -
vertical_ring_offset 0.73 -vertical_ring_width 3 -horizontal_strap_layer METAL5 -
horizontal_strap_width 3 -vertical_strap_layer METAL4 -vertical_strap_width 3
#@ commit_fp_group_block_ring
#@
#@ set_fp_rail_region_constraints -polygon {{1436 1660} {1436 1794} {1807 1794}
{1807 1660}}
#@ create_fp_group_block_ring -nets {VDD VSS} -horizontal_ring_layer METAL5 -
horizontal_ring_offset 0.78 -horizontal_ring_width 3 -vertical_ring_layer METAL4 -
vertical_ring_offset 0.73 -vertical_ring_width 3 -horizontal_strap_layer METAL5 -
horizontal_strap_width 3 -vertical_strap_layer METAL4 -vertical_strap_width 3
#@ commit_fp_group_block_ring
#@
#@ set_fp_rail_region_constraints -polygon {{1500 1300} {1500 1663} {1807 1663}
{1807 1300}}
#@ create_fp_group_block_ring -nets {VDD VSS} -horizontal_ring_layer METAL5 -
horizontal_ring_offset 0.78 -horizontal_ring_width 3 -vertical_ring_layer METAL4 -
vertical_ring_offset 0.73 -vertical_ring_width 3 -horizontal_strap_layer METAL5 -
horizontal_strap_width 3 -vertical_strap_layer METAL4 -vertical_strap_width 3
#@ commit_fp_group_block_ring
#@
#@ set_fp_rail_region_constraints -polygon {{1366 366} {1366 776} {1794 776}
{1794 366}}
#@ create_fp_group_block_ring -nets {VDD VSS} -horizontal_ring_layer METAL5 -
horizontal_ring_offset 0.78 -horizontal_ring_width 3 -vertical_ring_layer METAL4 -
vertical_ring_offset 0.73 -vertical_ring_width 3 -horizontal_strap_layer METAL5 -
horizontal_strap_width 3 -vertical_strap_layer METAL4 -vertical_strap_width 3
#@ commit_fp_group_block_ring
#@
#@ set_fp_rail_region_constraints -remove
#@ # -- End source

```

17- Synthesize power network:

```

source scripts/pns.tcl
#@ ### Complete the power grid structure: Ring around individual PLL macro,
vertical/horizontal straps, and core ring:
#@
#@ set_fp_rail_constraints -add_layer -layer METAL5 -direction horizontal -max_strap
24 -min_strap 2 -min_width 2 -max_width 4 -spacing 0.6

```

```

#@ set_fp_rail_constraints -add_layer -layer METAL4 -direction vertical -max_strap 24
-min_strap 2 -min_width 2 -max_width 4 -spacing 0.6
#@ set_fp_rail_constraints -set_ring -horizontal_ring_layer { METAL3 } -
vertical_ring_layer { METAL2 } -ring_max_width 12 -ring_min_width 10 -extend_strap
core_ring
#@
#@ set_fp_block_ring_constraints -add -horizontal_layer METAL5 -horizontal_width 3 -
horizontal_offset 0.600 -vertical_layer METAL4 -vertical_width 3 -vertical_offset 0.600
-block_type instance -block {I_CLOCK_GEN/I_PLL_PCI} -net {VDD VSS}
#@ set_fp_rail_constraints -set_global -no_routing_over_hard_macros
#@ synthesize_fp_rail -nets {VDD VSS} -voltage_supply 1.32 -synthesize_power_plan
-power_budget 350 -pad_masters { pv0i pvdi }
#@ load_fp_rail_map -nets VDD -type IR -min 0.000000 -max 0.000000 -directory
./pna_output

```

-- End

18- Build the power network:

```
commit_fp_rail
```

19- Hook up power pins on all macros:

```

preroute_instances
preroute_standard_cells -fill_empty_rows -reomve _floating_pieces

```

20- Prevent standard cell placement under PNS layer:

```

set_pnet_options -complete "METAL4 METAL5"
create_fp_placement -timing_driven -no_hierarchy_gravity

```

21- Perform global routing:

```
route_zrt_global
```

22- Fix any timing violations:

```
optimize_fp_timing -fix_design_rule
```

CHAPTER 5: Placement, Clock Tree Synthesize and Routing Using Zroute

In this chapter, we go through the processes of Placement, Clock Tree Synthesize and Routing using Zroute.

5.1 Placement

The placement phase involves several key steps:

- Setup steps to control placement, DFT and power.
- Placement and optimization
- Incremental refinement to improve congestion and/or setup timing

1- Open design library and design cell

2- Apply timing and optimization controls:

```
source scripts/opt_ctrl.tcl
```

```
#@ # -----
#@ # General timing and optimization control settings
#@ # -----
#@ set_app_var timing_enable_multiple_clocks_per_reg true
#@ set_app_var case_analysis_with_logic_constants true
#@ set_app_var physopt_delete_unloaded_cells false
#@ set_app_var physopt_power_critical_range 0.8
#@ set_app_var physopt_area_critical_range 0.8
#@ set_app_var enable_recovery_removal_arcs true
#@
#@ # set_false_path from <clock_name> -to <clock_name>
#@ set_fix_multiple_port_nets -all -buffer_constants
#@ set_auto_disable_drc_nets -constant false
#@ set_timing_derate -max -early 0.95
#@ # set_dont_use <off_limit_cells>
#@ # set_prefer -min <hold_fixing_cells>
#@ set_max_area 0
#@ group_path -name INPUTS -from [all_inputs]
#@ group_path -name OUTPUTS -to [all_outputs]
#@ group_path -name COMBO -from [all_inputs] -to [all_outputs]
#@ set_ideal_network [all_fanout -flat -clock_tree]
#@ # set_cost_priority {max_transistion max_delay}
```



```
#@ # -- End source ./scripts/opt_ctrl.tcl
```

3-Apply Non-default routing rules:

```
source scripts/ndr.tcl
```

```
#@ define_routing_rule CLOCK_DOUBLE_SPACING -spacings {METAL3 0.42  
METAL4 0.63 METAL5 0.82}
```

```
#@
```

```
#@ set_clock_tree_options -routing_rule CLOCK_DOUBLE_SPACING -layer_list  
{METAL3 METAL5} -use_default_routing_for_sinks 1
```

```
#@
```

```
#@ # -- End source
```

4- Enable low-power placement (LPP) dynamic power optimization:

```
set_power_options -low_power_placement true
```

5-Run Placement:

```
place_opt -area_recovery -optimize_dft -power
```

6-Enable gate-level dynamic power optimization (GLPO):

```
set_power_options -dynamic true
```

7-Perform incremental logic optimization:

```
psynopt -area_recovery -power
```

5.2 Clock Tree Synthesize

The Clock Tree Synthesize phase involves several key steps:

- Setup steps to control CTS
- Optional Pre-CTS power optimization
- Clock Tree Synthesize
- Timing Optimization\
- Routing of clock nets

Figure 5.1 shows the initial clock tree that a single clock source drives all clock pins with no buffers.

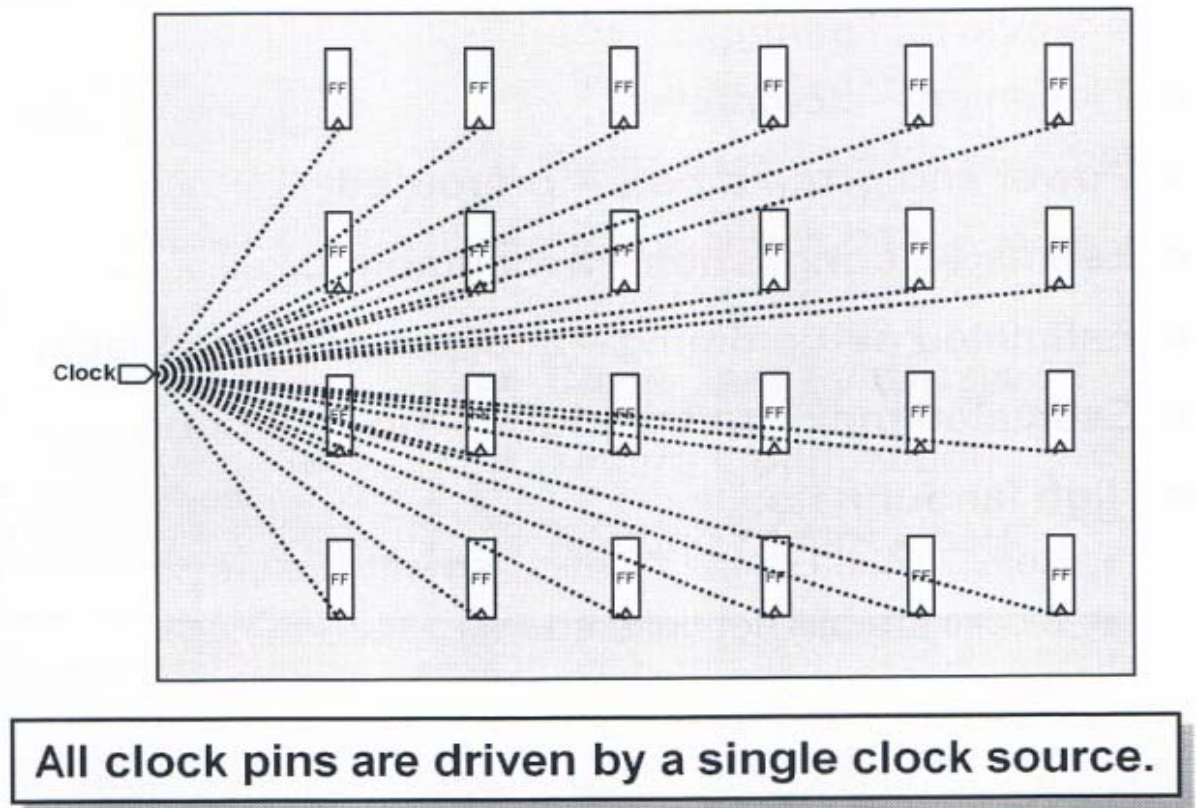


Figure 5.1 - Clock tree Synthesis – Initial Clock [21]

1-Apply the CTS setup script:

```
source ../scripts/cts_setup.tcl
```

```
##@ #####
##@ # Preparing for Clock Tree Synthesis #
##@ #####
##@
##@ set_clock_tree_exceptions -stop_pins {I_SDRAM_TOP/I_SDRAM_IF/sd_mux_*/S}
##@
##@ set_clock_tree_options -target_skew 0.1
##@
##@ set_clock_uncertainty 0.1 [all_clocks]
##@
##@ reset_clock_tree_references
##@ set_clock_tree_references -references {bufbd1 bufbd2 bufbd4 bufbd7 bufbdf}
##@
##@ define_routing_rule CLOCK_DOUBLE_SPACING -spacings {METAL3 0.42
METAL4 0.63 METAL5 0.82}
##@
##@ report_routing_rule CLOCK_DOUBLE_SPACING
##@
##@ set_clock_tree_options -routing_rule CLOCK_DOUBLE_SPACING -layer_list
{METAL3 METAL5} -use_default_routing_for_sinks 1
##@
##@ report_clock_tree -settings
##@
##@ check_physical_design -stage pre_clock_opt -no_display
##@ source -encrypted /usr/synopsys/E-2010.12-SP2-
ICC/auxx/syn/psyn/analyze_physical_design.tcl.e

##@ analyze_physical_design_internal {pre_clock_opt} {} 0 0 {} {} {0}

##@
##@ set_delay_calculation -clock_arnoldi
##@
##@ check_clock_tree
##@ # -- End source ../scripts/cts_setup.tcl
```

2-Configure CTS to use non-default routing:

```
source ../scripts/ndr.tcl
```

```
#@ define_routing_rule CLOCK_DOUBLE_SPACING -spacings {METAL3 0.42  
METAL4 0.63 METAL5 0.82}  
#@  
#@ set_clock_tree_options -routing_rule CLOCK_DOUBLE_SPACING -layer_list  
{METAL3 METAL5} -use_default_routing_for_sinks 1  
#@  
#@ # -- End source ../scripts/ndr.tcl
```

3-Perform the following check prior to running CTS:

```
check_physical_design -stage pre_clock_opt -display
```

4-Set the delay calculation to *Arnoldi* that is more accurate. (Instead of the default *Elmore*-based delay model)

```
set_delay_calculation -clock_arnoldi
```

5-Synthesize all clock trees, without any timing optimization or routing:

```
clock_opt -only_cts -no_clock_route
```

6-Enable hold time fixing:

```
set_fix_hold [all_clocks]
```

7-Perform post CTS timing, area and scan chain optimization, without clock routing:

```
clock_opt -only_psyn -area_recovery -optimize_dft -no_clock_route
```

8- Route the clocks:

```
route_zrt_group -all_clock_nets -reuse_existing_global_route true
```

5.3 Routing Using Zroute

Routing is done using the `route_opt` command.
`route_opt` performs the following:

- Global Routing
- Track Assignment
- Detail Routing

Figure 5.2 shows the basic Zroute flow, which includes clock routing, signal routing, DFM optimizations, and route verification.

Zroute has five routing engines: global routing, track assignment, detail routing, ECO routing, and routing verification. You can invoke global routing, track assignment, and detail routing by using the `route_opt` core command; by using task-specific commands; or by using an automatic routing command.

Zroute includes the following main features.

- Multithreading on multicore hardware for all routing steps, including global routing, track assignment, and detail routing.
- A realistic connectivity model where Zroute recognizes electrical connectivity if the rectangles touch; it does not require the center lines of wires to connect.
- A dynamic maze grid that permits Zroute to go off-grid to connect pins, while retaining the speed advantages of gridded routers.
- A polygon manager, which allows Zroute to recognize polygons and to understand that design rule checks (DRCs) are aimed at polygons.
- Concurrent optimization of design rules, antenna rules, wire optimization, and via optimization during detail routing.
- Concurrent redundant via insertion during detail routing.
- Support for soft rules built into global routing, track assignment, and detail routing.
- Timing- and crosstalk-driven global routing, track assignment, detail routing, and ECO routing.
- Intelligent design rule handling, including merging of redundant design rule violations and intelligent convergence.
- Net group routing with layer constraints and nondefault routing rules.
- Clock routing.
- Route verification.
- Optimization for DFM and design-for-yield (DFY) using a soft rule approach. [11]

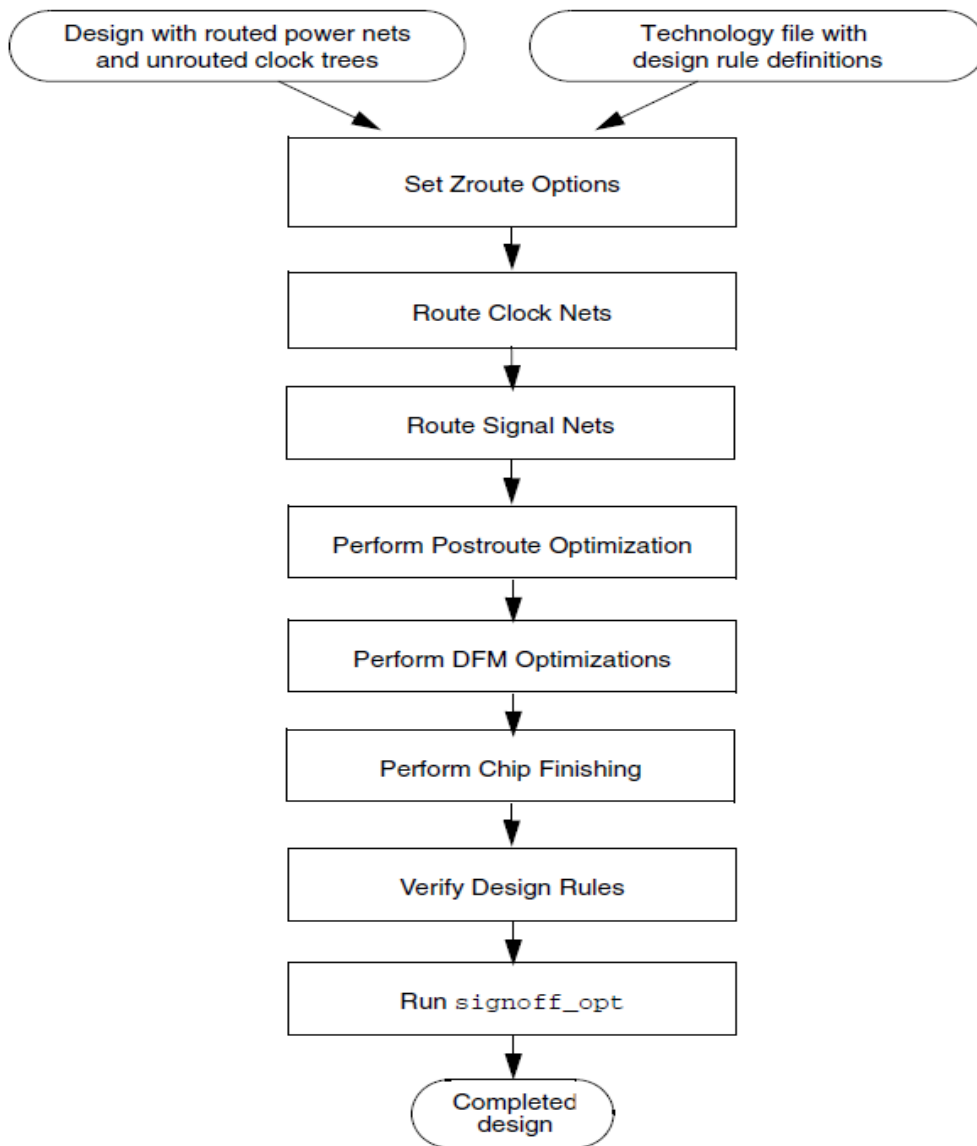


Figure 5.2 - Basic Zroute Flow [11]

1-Load the common stings required for routing phase:

```
source ../scripts/common_optimization_settings_icc.tcl
```

```
#@
#####
#####
#@ # Version: C-2009.06 (Jun 29th, 2009)
#@ # Copyright (C) 2007-2009 Synopsys, Inc. All rights reserved.
#@
#####
#####
#@
#@ echo "\tLoading :\t [info script]"
#@ ## Optimization Common Session Options - set in all sessions
#@
#@ # -----
#@ # General Optimization
#@ # enable the recovery and removal timing checks
#@ # -----
#@
#@ set_app_var enable_recovery_removal_arcs true
#@ set_app_var timing_enable_multiple_clocks_per_reg true
#@ set_app_var case_analysis_with_logic_constants true
#@ set_fix_multiple_port_nets -all -buffer_constants
#@ set_auto_disable_drc_nets -constant false
#@ set_timing_derate -max -early 0.95
#@ # set_dont_use <off_limit_cells>
#@ # set_prefer -min <hold_fixing_cells>
#@ set_max_area 0
#@ group_path -name INPUTS -from [all_inputs]
#@ group_path -name OUTPUTS -to [all_outputs]
#@ group_path -name COMBO -from [all_inputs] -to [all_outputs]
#@ # set_ideal_network [all_fanout -flat -clock_tree]
#@ # set_cost_priority {max_transistion max_delay}
#@
#@ # -----
#@ ## Fix the locations of the hard macros
#@ # -----
#@ if {[all_macro_cells] != ""} {
#@   set_dont_touch_placement [all_macro_cells]
```

```

#@ }
#@
#@ # -----
#@ # Timing and optimization control variable settings
#@ # -----
#@ set_app_var physopt_delete_unloaded_cells false
#@ set_app_var physopt_power_critical_range 0.4
#@ set_app_var physopt_area_critical_range 0.4
#@ set_app_var enable_recovery_removal_arcs true
#@
#@ ## Turn on leakage
#@ set_power_options -leakage true
#@ ## Dynamic power opto throughout the flow
#@ set_power_options -dynamic false -low_power_placement false
#@
#@ set_operating_conditions      -max cb13fs120_tsmc_max      -max_library
cb13fs120_tsmc_max      -min cb13fs120_tsmc_min      -min_library
cb13fs120_tsmc_min
#@
#@ ## End of Common Optimization Session Options
#@ # -- End source ../scripts/common_optimization_settings_icc.tcl

source ../scripts/common_placement_settings.tcl

#@
#####
#####
#@ # Version: C-2009.06 (Jun 29th, 2009)
#@ # Copyright (C) 2007-2009 Synopsys, Inc. All rights reserved.
#@
#####
#####
#@
#@
#@ echo "\tLoading :\t [info script]"
#@
#@ # Placement Common Session Options - set in all sessions
#@
#@ ## Set Min/Max Routing Layers
#@ #if { $MAX_ROUTING_LAYER != "" } {set_ignored_layers -max_routing_layer
$MAX_ROUTING_LAYER}

```



```

#@ #if { $MIN_ROUTING_LAYER != "" } {set_ignored_layers -min_routing_layer
$MIN_ROUTING_LAYER}
#@
#@ # -----
#@ # Placement keepout variable settings
#@ # -----
#@ set_app_var physopt_hard_keepout_distance 5
#@ set_app_var placer_soft_keepout_channel_width 15
#@
#@ ## Set PNET Options to control cel placement around P/G straps
#@ remove_pnet_options
#@ set_pnet_options -partial {METAL2 METAL3 METAL4}
#@ report_pnet_options
#@
#@ ## Improved congestion analysis by using Global Route info
#@ # echo "SCRIPT-Info : Enabling Global Gouter during placement"
#@ # set_app_var placer_enable_enhanced_router true
#@
#@ ## it is recommended to use the default of the tool
#@ ## in case it needs to change ( e.g. for low utilization designs), use the command
below :
#@ # set_congestion_options -max_util 0.85
#@ # -- End source ../scripts/common_placement_settings.tcl

```

```

source ../scripts/common_post_cts_timing_settings.tcl

```

```

#@
#####
#####
#@ # Version: C-2009.06 (Jun 29th, 2009)
#@ # Copyright (C) 2007-2009 Synopsys, Inc. All rights reserved.
#@
#####
#####
#@
#@
#@ echo "\tLoading :\t\t [info script]"
#@
#@
#@ ## Enabling CRPR - CRPR is usually used with timing derate (bc_wc) and with
OCV

```

```

#@ set_app_var timing_remove_clock_reconvergence_pessimism true
#@
#@
#@ ## Set Area Critical Range
#@ ## Typical value: 5 percent of critical clock period
#@ #if {$AREA_CRITICAL_RANGE_POST_CTS != ""} {set_app_var
physopt_area_critical_range $AREA_CRITICAL_RANGE_POST_CTS}
#@
#@
#@ ## Set Power Critical Range
#@ ## Typical value: 5 percent of critical clock period
#@ #if {$POWER_CRITICAL_RANGE_POST_CTS != ""} {set_app_var
physopt_power_critical_range $POWER_CRITICAL_RANGE_POST_CTS}
#@
#@
#@ ## Hold fixing cells
#@ #if { $ICC_FIX_HOLD_PREFER_CELLS != ""} {
#@ #  remove_attribute $ICC_FIX_HOLD_PREFER_CELLS dont_use
#@ #  remove_attribute $ICC_FIX_HOLD_PREFER_CELLS dont_touch
#@ #  set_prefer -min $ICC_FIX_HOLD_PREFER_CELLS
#@ #}
#@
#@ # -- End source ../scripts/common_post_cts_timing_settings.tcl

source ../scripts/common_route_si_settings_zrt_icc.tcl

#@
#####
#####
#@ # Version: C-2009.06 (Jun 29th, 2009)
#@ # Copyright (C) 2007-2009 Synopsys, Inc. All rights reserved.
#@
#####
#####
#@
#@ echo "\tLoading :\t\t [info script]"
#@
#@ #####
#@ #      TIMING OPTIONS      #
#@ #####

```

```

#@
#@ ## By default, Xtalk Delta Delay is enabled for all flows
#@ set_si_options -delta_delay true -route_xtalk_prevention true -
route_xtalk_prevention_threshold 0.25
#@
#@ ## For the QoR flow, we also enable min_delta_delay
#@ set_si_options -min_delta_delay true
#@
#@ #####
#@ # MAX_TRAN FIXING #
#@ #####
#@ ## From 2006.06-SP4 onwards, route_opt will NOT fix nor report Delta Max
#@ ## Tran violations. Hence all max_tran violations exclude the portion
#@ ## that is introduced by Xtalk.
#@ ## If you want to change this behavior, and fix max_transition violations
#@ ## including these caused by Xtalk, please use the switch -max_transition_mode
#@ ## in set_si_options. Keep in mind that you can expect a runtime hit of up
#@ ## to 2x in DRC fixing during route_opt.
#@
#@ # set_si_options -delta_delay true # -route_xtalk_prevention true #
-route_xtalk_prevention_threshold 0.25 # -max_transition_mode total_slew
#@
#@ #####
#@ # ADVANCED TIMING FEATURES #
#@ #####
#@
#@ ## if static noise (aka glitches) needs to be reduced, please use the extra options
below :
#@ # set_si_options -delta_delay true # -static_noise true # -
static_noise_threshold_above_low 0.35 # -static_noise_threshold_below_high
0.35 # -route_xtalk_prevention true # -
route_xtalk_prevention_threshold 0.25
#@
#@
#@ set_delay_calculation -arnoldi
#@ set ICC_FULL_ARNOLDI FALSE
#@ if {$ICC_FULL_ARNOLDI} {
#@ echo "SCRIPT-Info : Enabling Arnoldi for all nets of the design"
#@ set rc_rd_more_than_rnet_arnoldi_threshold 100000000
#@ }
#@

```

```

#@ ## if you want to enable Timing Windows during XDD calculation, please use :
#@ # set_si_options -timing_window true
#@
#@ #####
#@ #      ZROUTE OPTIONS      #
#@ #####
#@
#@ ##### MultiThreading - extraction, MCMM timing, ZRoute
#@ set ICC_NUM_CORES 1
#@ set_host_options -max_cores $ICC_NUM_CORES
#@
#@ ## Zroute Global route options
#@ # set_route_zrt_global_options
#@
#@ ## Zroute Track assign options
#@ # set_route_zrt_track_options
#@
#@ ## Zroute Detail route options
#@ # set_route_zrt_detail_options
#@
#@ ## Set Area Critical Range
#@ ## Typical value: 3-4 percent of critical clock period
#@ #set_app_var physopt_area_critical_range 0.32
#@
#@ ## Set Power Critical Range
#@ ## Typical value: 3-4 percent of critical clock period
#@ #set_app_var physopt_power_critical_range 0.32
#@
#@ #####
#@ #      ROUTE_OPT ONLY OPTIONS      #
#@ #####
#@
#@ ## Disable fanout opto during route_opt. Max fanout has to be taken care of earlier in
the flow.
#@ set_app_var psyn_onroute_disable_fanout_drc true
#@
#@ ##default is false - set to skip second report_qor in route_opt
#@ set_app_var routeopt_skip_report_qor true
#@
#@ # set_app_var routeopt_xtalk_reduction_setup_threshold 0.10
#@

```

```

#@ set ICC_DBL_VIA TRUE
#@ set ICC_DBL_VIA_FLOW_EFFORT "LOW"
#@ if { $ICC_DBL_VIA } {
#@   define_zrt_redundant_vias -from_via { VIA12A VIA12B VIA23 VIA34 VIA45
VIA56 } -to_via { VIA12A VIA12B VIA23 VIA34 VIA45 VIA56 } -
to_via_x_size { 1 1 1 1 1 1 } -to_via_y_size { 2 2 2 2 2 2 }
#@
#@ if { $ICC_DBL_VIA_FLOW_EFFORT != "LOW" } {
#@   set_route_zrt_detail_options -optimize_wire_via_effort_level high ;#low is
default
#@   set_route_zrt_common_options -concurrent_redundant_via_mode reserve_space
#@ }
#@ if { $ICC_DBL_VIA_FLOW_EFFORT == "HIGH" } {
#@   #low is default: low|medium|high
#@   set_route_zrt_common_options -concurrent_redundant_via_effort_level medium
#@   #set_route_zrt_common_options -eco_route_concurrent_redundant_via_mode
reserve_space
#@   #low is default: low|medium|high
#@   #set_route_zrt_common_options -
eco_route_concurrent_redundant_via_effort_level low
#@ }
#@ }
#@
#@ set ICC_FIX_ANTENNA 0
#@ set ANTENNA_RULES_FILE ""
#@
#@ if { $ICC_FIX_ANTENNA } {
#@
#@ #####
#@ #   ANTENNA JOGGING FIXING   #
#@ #####
#@
#@ if {[file exists [which $ANTENNA_RULES_FILE]]} {
#@   set_route_zrt_detail_options -antenna true
#@   source -echo $ANTENNA_RULES_FILE
#@ } else {
#@   echo "SCRIPT-Info : Antenna rules file does not exist"
#@   echo "SCRIPT-Info : Turning off antenna fixing"
#@   set_route_zrt_detail_options -antenna false
#@ }
#@
#@

```

```

#@ } else {
#@     echo "SCRIPT-Info : Turning off antenna fixing"
#@     set_route_zrt_detail_options -antenna false
#@ }
#@
#@ # -- End source ../scripts/common_route_si_settings_zrt_icc.tcl

```

12- Enable “concurrent” redundant via insertion:

```

set_route_zrt_common_options -post_detail_route_redundant_via_insertion medium
set_route_zrt_detail_options -optimize_wire_via_effort_level medium

```

13- Perform initial routing, which includes *global routing*, *track assignment* and *detail routing*:

```

route_opt -initial_route_only

```

14- Perform post-initial route optimization:

```

route_opt -skip_initial_route -power

```

15-Run incremental route_opt:

```

route_opt -incremental

```

Chapter 6: Analysis of Results and Conclusion

6.1 Analysis of Results

In DC Compiler and IC Compiler, Cb13fs120_tsmc_max 90nm library is used to optimize the design for smallest area, fastest speed and lowest power.

Without low power techniques, we achieve following results:

Cell Internal Power	=	87.6532 mW	
Net Switching Power	=	6.9432 mW	-----
Total Dynamic Power	=	96.2358 mW	

Cell Leakage Power = 145.3489 uW

With using low power techniques, we achieve following results:

Cell Internal Power	=	49.3431 mW	
Net Switching Power	=	2.3906 mW	-----
Total Dynamic Power	=	51.7338 mW	

Cell Leakage Power = 89.1487 uW

It is clear from the above results that the Clock gating and Multi Vt library which are implemented in design, reduced dynamic power to 51.7338 mW and leakage power to 89.1487 uW.

6.2 Conclusion

The main objectives of this project were to implement RTL to GDSII flow using company standard EDA tools DC Compiler and IC Compiler from Synopsys Inc. with low power methodology. Complete ASIC design is successfully implemented and power optimization and DFT methodologies like logic scan, JTAG boundary scan were implemented using Power Compiler and DFT Compiler respectively. In addition, main emphasis is given to physical implementation of standard cell ASICs using IC Compiler. This part covers the detailed back-end design flow including analyzing and fixing of congestion issues, timing closure and DRC and LVS violations.

This project provided the strong foundation to excel career as an ASIC designer, backed-up with sufficient knowledge and exposure to complete ASIC design flow, with hands on experience of the Synopsys EDA tools.

REFERENCES

1. <http://www.siliconfareast.com/asic.htm>
2. www.omniglot.com/info-articles/dallas/application_specific_integrated_circuits.html
3. MICHAEL JOHN SEBASTIAN SMITH [June 1997], "Application-Specific Integrated Circuits", Addison-Wesley Publishing Company, VLSI Design Series.
4. <http://www.vlsichipdesign.com/index.php/Chip-Design-Articles/analogy-of-vlsi-and-building-architecture-a-thought-process.html>
5. Himanshu Bhatnagar, "Advanced ASIC chip Synthesis Using Synopsys Design Compiler, Physical Compiler and Primetime" 2004.
6. Michael Keating, David Flynn, Robert Aitken, Alan Gibbons, kaijian Shi, "Low Power Methodology Manual"
7. Vishwani D. Agrawal, Srivaths Ravi, "Low-Power Design and Test", July 2007.
8. ASIC Design Flow Tutorial Using Synopsys Tools By Hima Bindu Kommuru, Hamid Mahmoodi, Nano-Electronics & Computing Research Lab, School of Engineering, San Francisco State University San Francisco, CA
9. Synopsys Design Compiler User Guide Version D-2010.03, March 2010
10. Advanced ASIC Chip Synthesis 2ed from www.scribd.com
11. Synopsys IC Compiler User Guide Version D-2010.03, March 2010
12. <http://www.utdallas.edu/~zhoud/Lecture1.pdf>
13. Synopsys Low-Power Flow User Guide, Version F-2011.09, September 2011
14. Power Compiler User Guide, Version F-2011.09, September 2011
15. Timing Constraint and Optimization User Guide, Version F-2011.09, September 2011
16. IC Compiler User Guide Version A-2007.12, December 2007
17. ASIC/IC Design-for-Test Process Guide, Software Version 8.6_1, Mentor Graphics.
18. <http://semicon.sanyo.com/en/asic/user/cts.php>
19. http://en.wikipedia.org/wiki/Post-silicon_validation
20. <http://people.ee.duke.edu/~krish/teaching/Lectures/Testing.1.pdf>
21. Synopsys customer education services IC Compiler 1 student Guide 2012

APPENDIX A

1. RISC CHIP Floorplanned

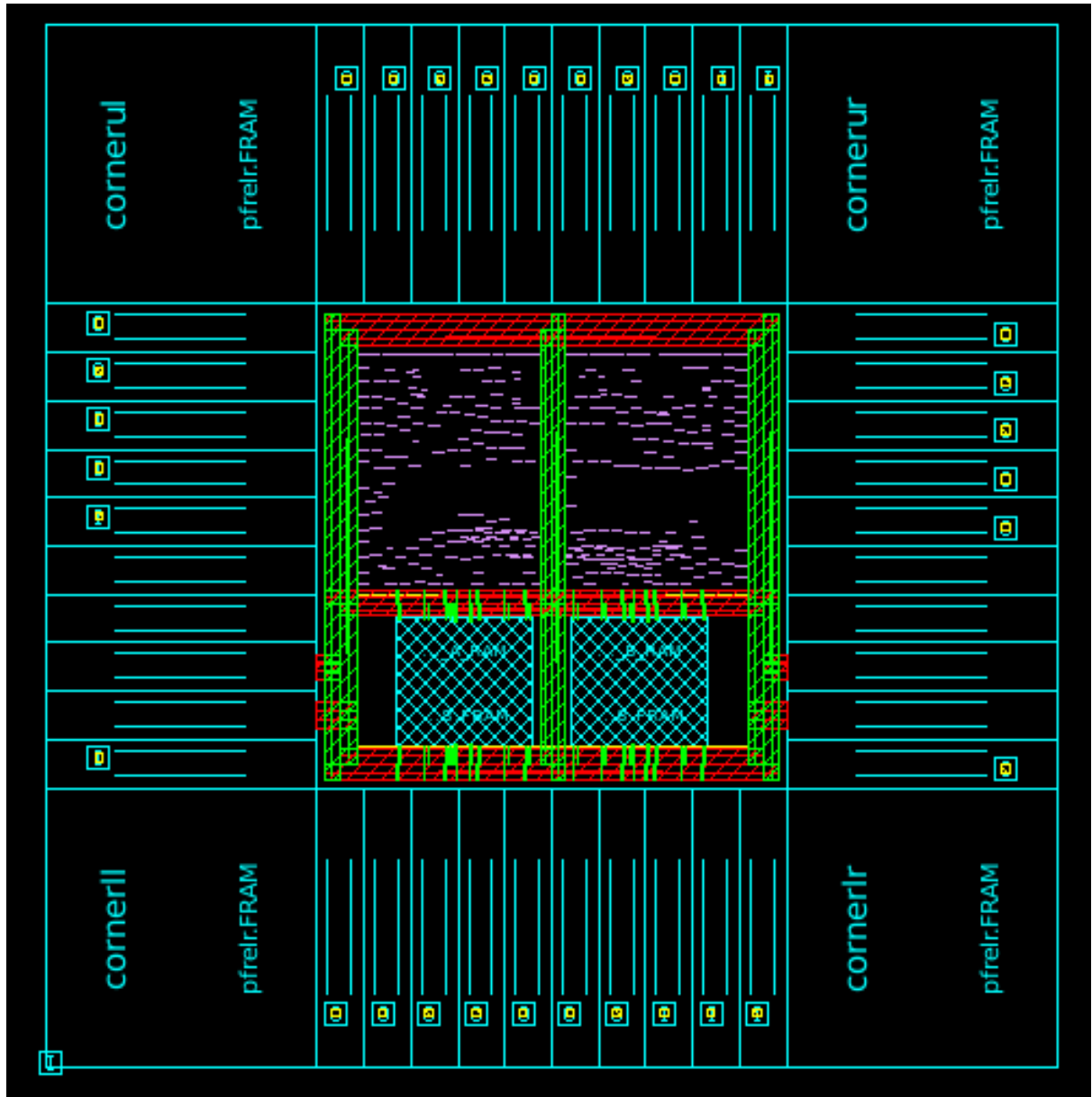


Figure A.1: RISC CHIP Floorplanned.

2. RISC CHIP Placed

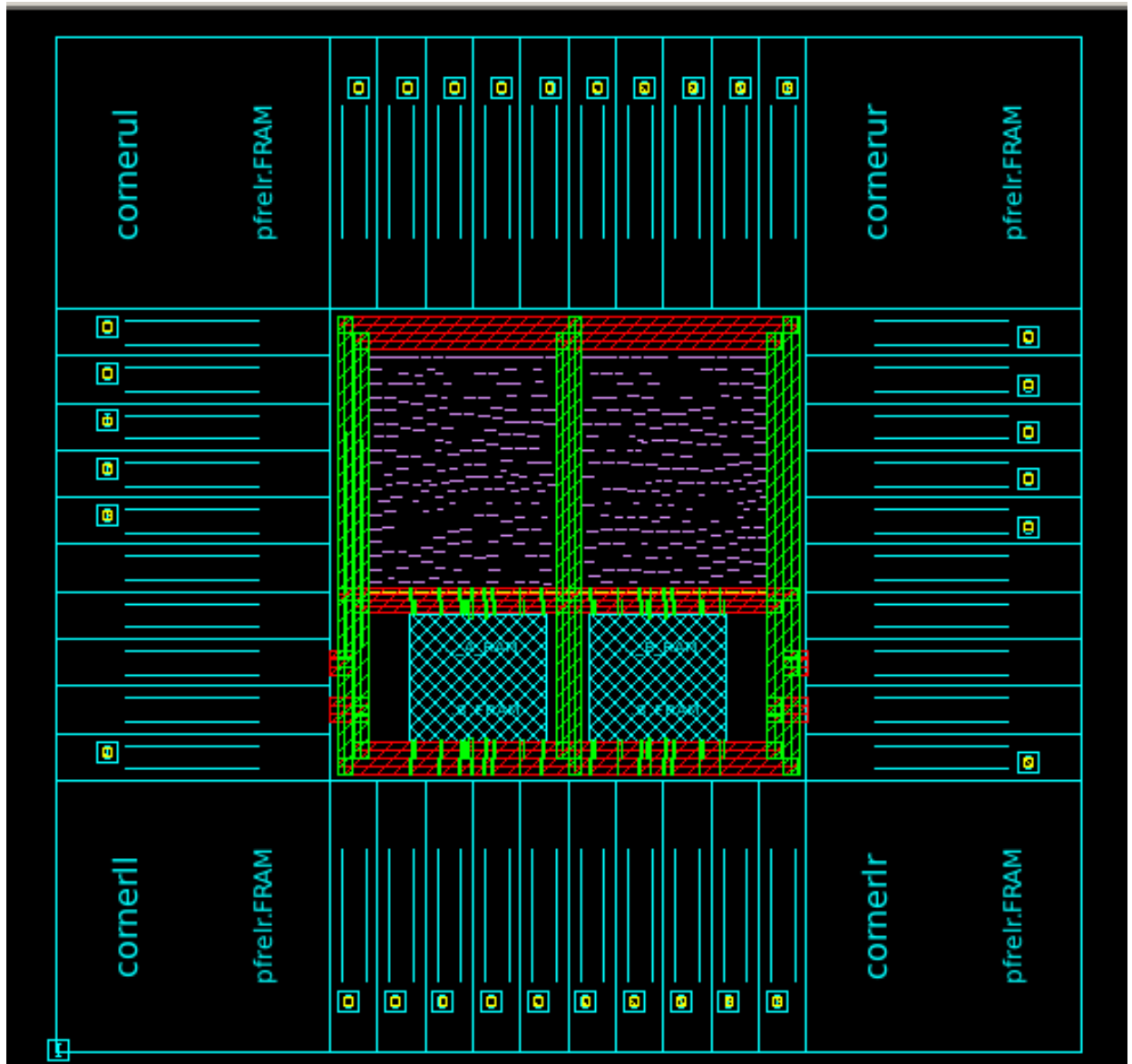


Figure A.2: RISC CHIP Placed

3. Initial Routing

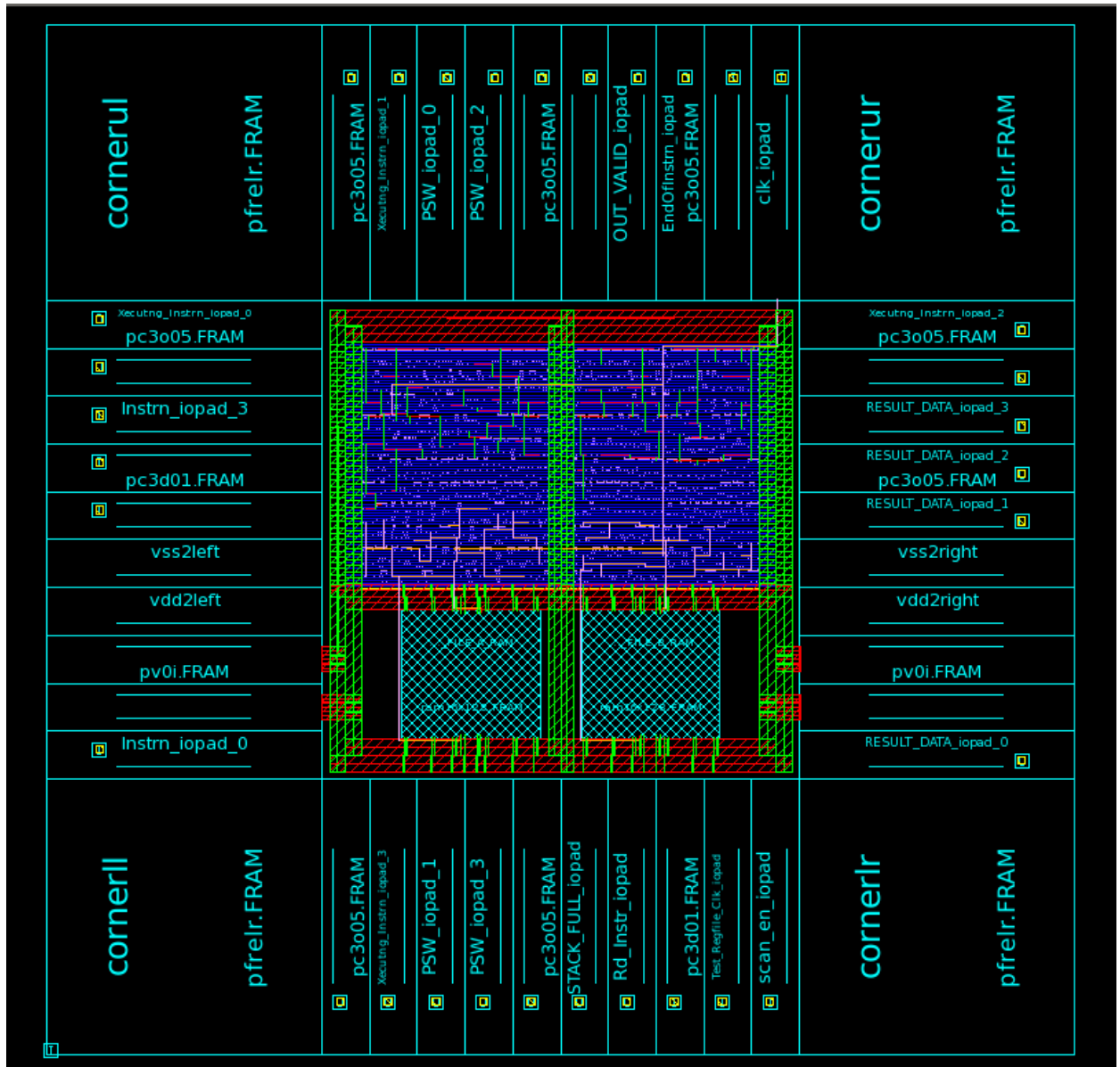


Figure A.3: Initial Routing

4. Power Density

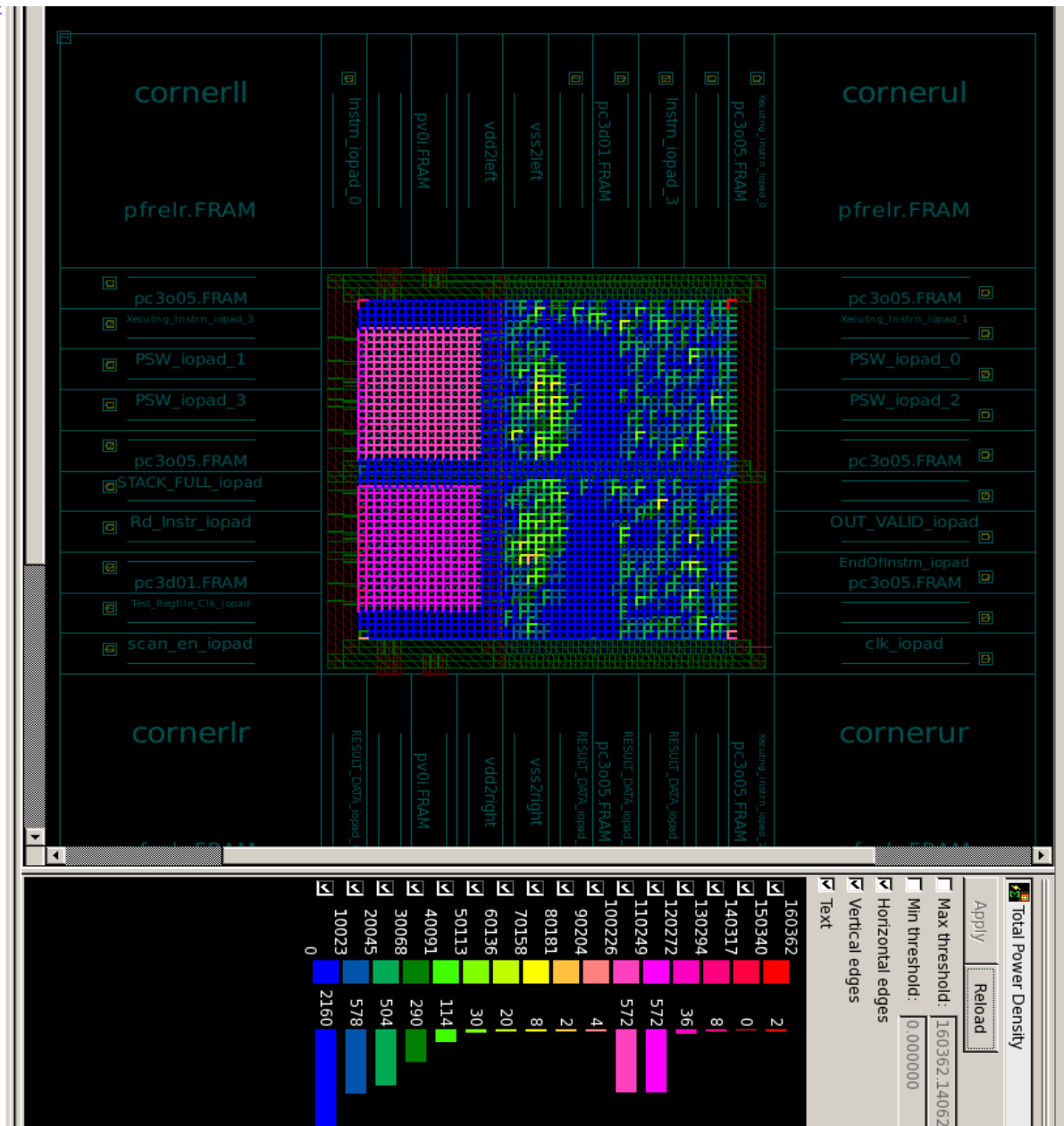


Figure A.4: Power Density

5. Clock Tree

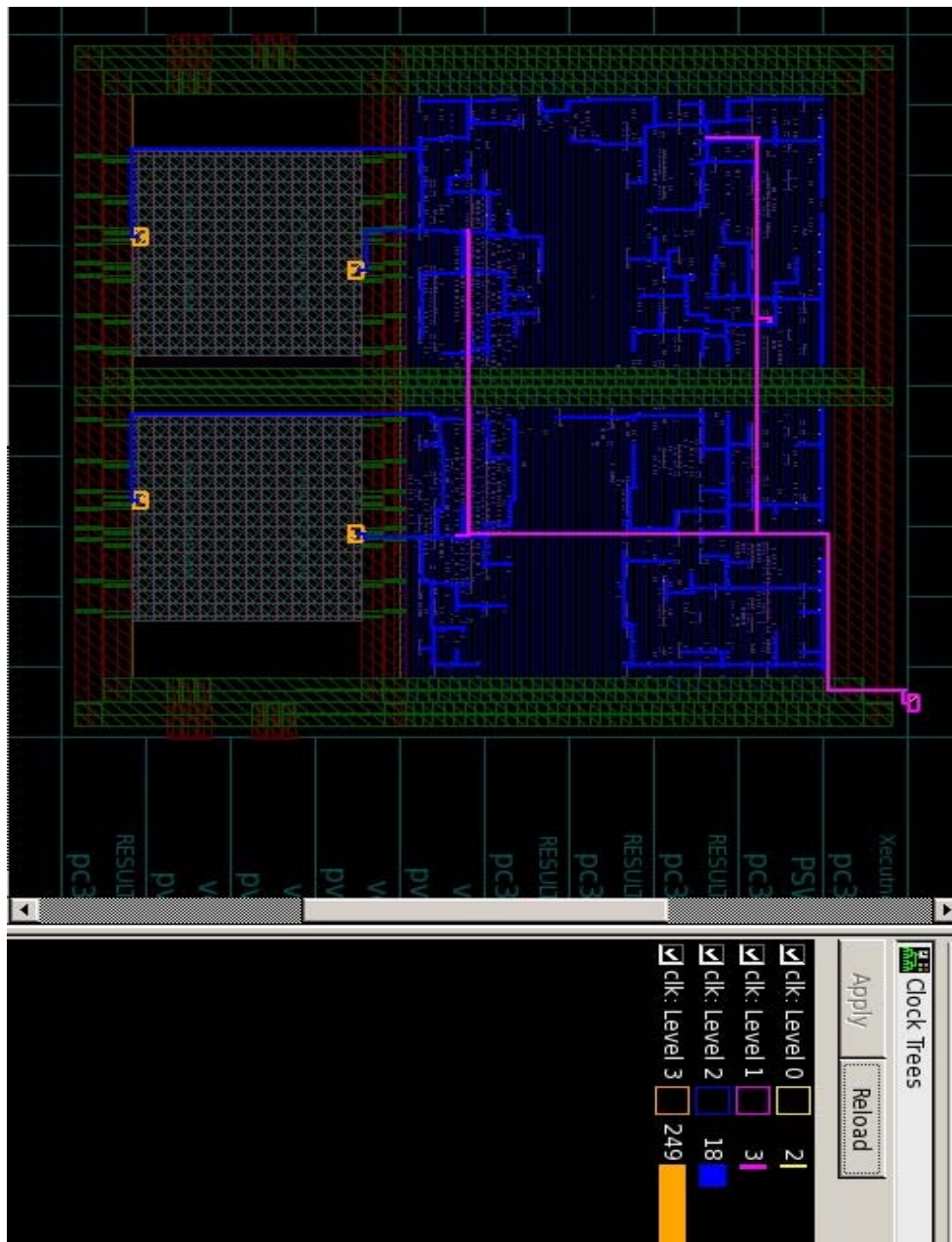


Figure A.5: Clock Tree

6. Detailed Routing

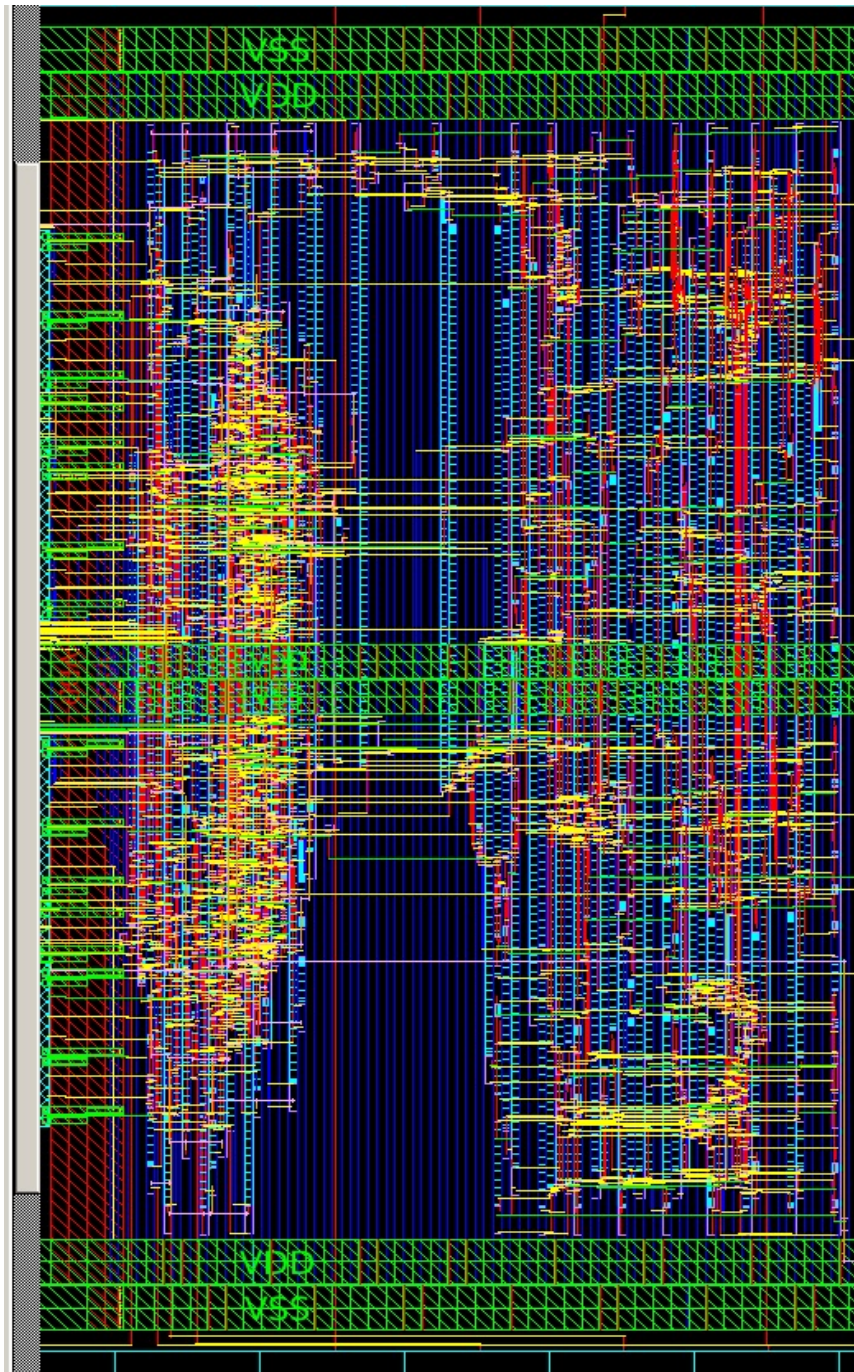


Figure A.6: Detailed Routing

7. Final Layout

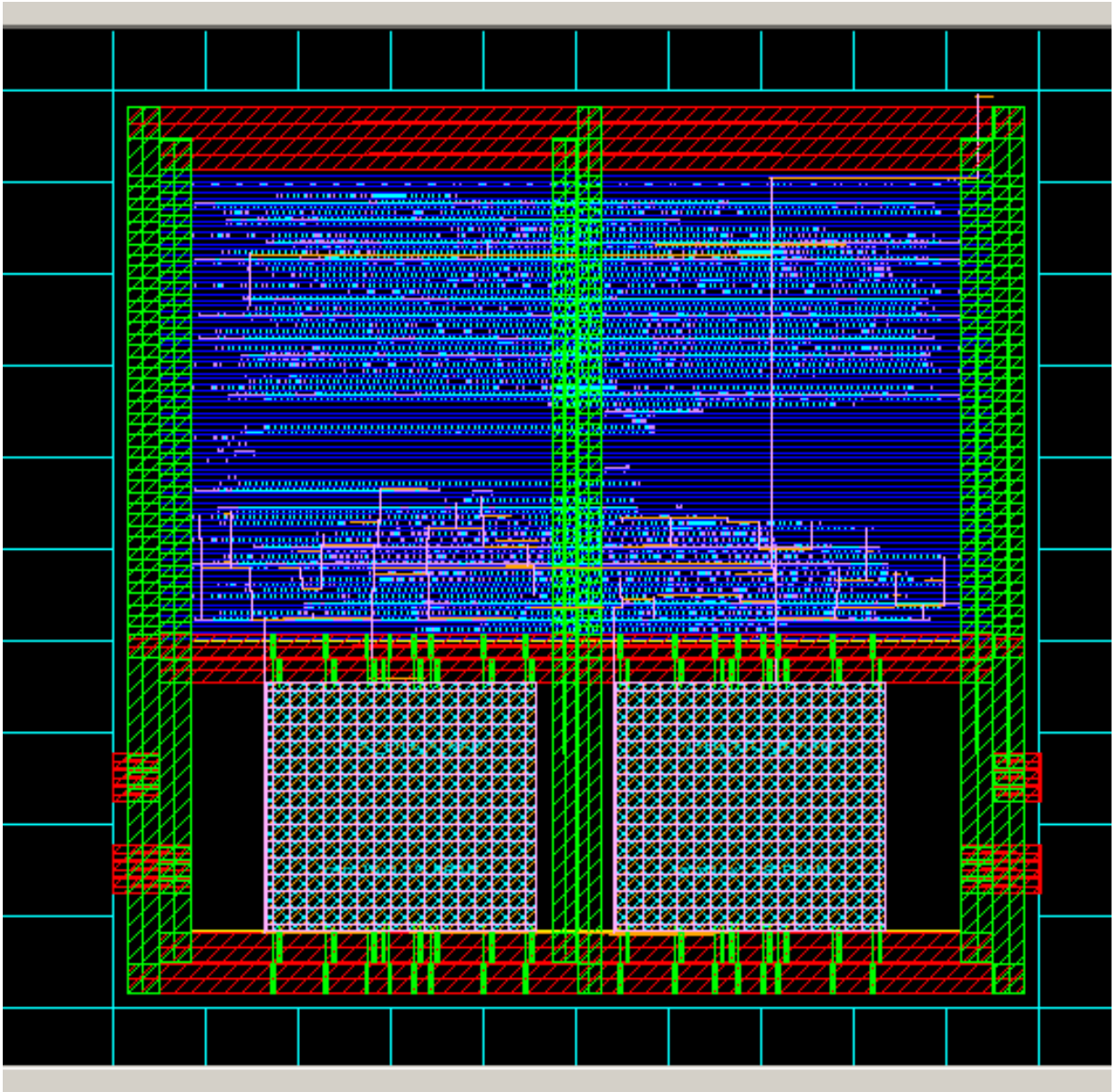


Figure A.7: Final Layout

APPENDIX B

1-Timing Report

Report : timing

-path full

-delay max

-max_paths 1

Design : RISC_CHIP

Version: E-2010.12-ICC-SP2

Date : Thu Nov 8 11:01:33 2012

* Some/all delay information is back-annotated.

Operating Conditions: cb13fs120_tsmc_max Library: cb13fs120_tsmc_max

Information: Percent of Arnoldi-based delays = 3.31%

Startpoint: Instrn[0] (input port clocked by clk)

Endpoint: I_RISC_CORE/I_INSTRN_LAT/Crnt_Instrn_2_reg_24_

(rising edge-triggered flip-flop clocked by clk)

Path Group: INPUTS

Path Type: max

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (ideal)	1.00	1.00
input external delay	4.00	5.00 r
Instrn[0] (in)	0.00	5.00 r
Instrn_iopad_0/CIN (pc3d01)	0.96 &	5.96 r
I_RISC_CORE/Instrn[24] (RISC_CORE)	0.00	5.96 r
I_RISC_CORE/I_INSTRN_LAT/Instrn[24] (INSTRN_LAT)	0.00	5.96 r
I_RISC_CORE/I_INSTRN_LAT/U115/Z (mx02d0)	0.19 &	6.15 r
I_RISC_CORE/I_INSTRN_LAT/Crnt_Instrn_2_reg_24_/D (sdnrq1)	0.00 &	6.15 r
data arrival time		6.15
clock clk (rise edge)	8.00	8.00

clock network delay (propagated)	0.00	8.00
clock uncertainty	-0.10	7.90
I_RISC_CORE/I_INSTRN_LAT/Crnt_Instrn_2_reg_24_/CP (sdnrq1)	0.00	7.90 r
library setup time	-0.22	7.68
data required time		7.68

data required time		7.68
data arrival time		-6.15

slack (MET)		1.52

Startpoint: I_RISC_CORE/I_REG_FILE/REG_FILE_A_RAM
(rising edge-triggered flip-flop clocked by clk)

Endpoint: RESULT_DATA[0]
(output port clocked by clk)

Path Group: OUTPUTS

Path Type: max

Point	Incr	Path

clock clk (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
I_RISC_CORE/I_REG_FILE/REG_FILE_A_RAM/CE2 (ram16x128)	0.00	0.00 r
I_RISC_CORE/I_REG_FILE/REG_FILE_A_RAM/O2[1] (ram16x128)	1.82	1.82 f
I_RISC_CORE/I_REG_FILE/RegPort_A[1] (REG_FILE)	0.00	1.82 f
I_RISC_CORE/RESULT_DATA[1] (RISC_CORE)	0.00	1.82 f
U39/ZN (nd02d1)	0.05 &	1.86 r
U32/Z (xr02d1)	0.25 &	2.11 f
U11/Z (xr02d1)	0.77 &	2.88 f
RESULT_DATA_iopad_0/PAD (pc3o05)		2.17 & 5.05 f
RESULT_DATA[0] (out)	0.00	5.05 f
data arrival time		5.05
clock clk (rise edge)	8.00	8.00
clock network delay (ideal)	1.00	9.00
clock uncertainty	-0.10	8.90
output external delay	-2.80	6.10

data required time	6.10
--------------------	------

data required time	6.10
data arrival time	-5.05

slack (MET)	1.05
-------------	------

Startpoint: I_RISC_CORE/I_DATA_PATH/Oprnd_B_reg_0_
(rising edge-triggered flip-flop clocked by clk)

Endpoint: I_RISC_CORE/I_ALU/Zro_Flag_reg
(rising edge-triggered flip-flop clocked by clk)

Path Group: clk

Path Type: max

Point	Incr	Path
clock clk (rise edge)	0.00	0.00
clock network delay (propagated)	0.00	0.00
I_RISC_CORE/I_DATA_PATH/Oprnd_B_reg_0_/CP (sdnrq1)	0.00	0.00 r
I_RISC_CORE/I_DATA_PATH/Oprnd_B_reg_0_/Q (sdnrq1)	0.35	0.35 r
I_RISC_CORE/I_DATA_PATH/Oprnd_B[0] (DATA_PATH)	0.00	0.35 r
I_RISC_CORE/I_ALU/Oprnd_B[0] (ALU)	0.00	0.35 r
I_RISC_CORE/I_ALU/U14/ZN (inv0d0)	0.23 &	0.58 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/B[0] (ALU_DW01_sub_0)	0.00	0.58 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U1/Z (an02d0)	0.20 &	0.78 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_1/CO (ad01d0)	0.26 &	1.04 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_2/CO (ad01d0)	0.26 &	1.31 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_3/CO (ad01d0)	0.26 &	1.57 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_4/CO (ad01d0)	0.26 &	1.84 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_5/CO (ad01d0)	0.26 &	2.10 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_6/CO (ad01d0)	0.26 &	2.36 f
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_7/CO (ad01d0)	0.26 &	2.63 f

I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_8/CO (ad01d0)	0.26 & 2.89 f		
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_9/CO (ad01d0)	0.26 & 3.16 f		
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_10/CO (ad01d0)	0.26 & 3.42 f		
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_11/CO (ad01d0)	0.26 & 3.69 f		
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_12/CO (ad01d0)	0.26 & 3.95 f		
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_13/CO (ad01d0)	0.26 & 4.22 f		
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_14/CO (ad01d0)	0.26 & 4.48 f		
I_RISC_CORE/I_ALU/sub_1_root_sub_92/U2_15/S (ad01d0)	0.34 & 4.81 f		
I_RISC_CORE/I_ALU/sub_1_root_sub_92/DIFF[15] (ALU_DW01_sub_0)	0.00 & 4.81 f		
I_RISC_CORE/I_ALU/U155/Z (aor222d1)	0.31 & 5.12 f		
I_RISC_CORE/I_ALU/U154/Z (aor221d1)	0.23 & 5.35 f		
I_RISC_CORE/I_ALU/U153/ZN (nr02d0)	0.09 & 5.44 r		
I_RISC_CORE/I_ALU/U47/Z (an04d0)	0.21 & 5.66 r		
I_RISC_CORE/I_ALU/U96/ZN (oai2222d1)	0.34 & 5.99 f		
I_RISC_CORE/I_ALU/U48/ZN (inv0d0)	0.06 & 6.05 r		
I_RISC_CORE/I_ALU/U6/Z (mx02d0)	0.17 & 6.22 r		
I_RISC_CORE/I_ALU/Zro_Flag_reg/D (sdcrq1)	0.00 & 6.22 r		
data arrival time	6.22		
clock clk (rise edge)	8.00	8.00	
clock network delay (propagated)		0.00	8.00
clock uncertainty	-0.10	7.90	
I_RISC_CORE/I_ALU/Zro_Flag_reg/CP (sdcrq1)		0.00	7.90 r
library setup time	-0.22	7.68	
data required time		7.68	

data required time		7.68	
data arrival time		-6.22	

slack (MET)		1.46	

2-Physical Design Report

Report : design

-physical

Design : RISC_CHIP

Version: E-2010.12-ICC-SP2

Date : Thu Nov 8 11:05:50 2012

***** P&R Summary

Date : Thu Nov 8 11:05:50 2012

Machine Host Name: eesun04.ecs.csun.edu

Working Directory: /home/users5/ma581955/lab1_data_setup

Library Name: RISC_CHIP

Cell Name: RISC_final.CEL;1

Design Statistics:

Number of Macro Cells:	2
Number of Module Cells:	8794
Number of Pins:	23156
Number of IO Pad Cells:	40
Number of IO Pins:	38
Number of Nets:	1360
Average Pins Per Net (Signal):	3.37073

Chip Utilization:

Total Macro Cell Area:	57868.43
Total Std Cell Area:	23203.35
Total Blockage Area:	95978.38
Total Pad Cell Area:	1307016.68
Core Size:	width 500.20, height 498.15; area 249174.63
Pad Core Size:	width 600.20, height 600.00; area 360120.00
Chip Size:	width 1291.33, height 1291.13; area 1667274.90
Std cells utilization:	15.15%
Cell/Core Ratio:	32.54%
Cell/Chip Ratio:	83.25%
Number of Cell Rows:	135

Master Instantiation:

MasterName	Type	InstCount
------------	------	-----------

=====			
feedth9	STD	4551	
feedth	STD	2415	
feedth3	STD	652	
mx02d0	STD	233	
sdnrq1	STD	221	
inv0d0	STD	92	
nr02d0	STD	64	
ad01d0	STD	60	
aor22d1	STD	56	
aor221d1	STD	51	
aor222d1	STD	42	
nr03d0	STD	40	
an02d0	STD	40	
xr02d1	STD	33	
aoi2222d1	STD	25	
nd04d0	STD	25	
sdcrq1	STD	24	
dl01d1	STD	21	
ah01d0	STD	20	
nd02d0	STD	15	
bufbd1	STD	14	
aor21d1	STD	14	
aoim22d1	STD	11	
xr03d1	STD	11	
an04d0	STD	7	
nr04d0	STD	6	
xn02d1	STD	6	
an03d0	STD	6	
bufbd7	STD	6	
aor211d1	STD	4	
nd02d1	STD	4	
or02d0	STD	4	
nd03d0	STD	3	
nd12d0	STD	3	
an12d1	STD	2	
aor31d1	STD	2	
an03d1	STD	1	
aon211d1	STD	1	
inv0d1	STD	1	
or03d0	STD	1	

nr03d1	STD	1	
or04d0	STD	1	
invbd4	STD	1	
oai2222d1	STD	1	
oan211d1	STD	1	
nr23d1	STD	1	
aoi21d1		STD	1
ram16x128	MACRO		2
pfeed00010	IO	36	
pc3o05	IO	23	
pc3d01	IO	9	
pfeed00005	IO	8	
pvda	IO	2	
pv0a	IO	2	
pvdI	IO	2	
pv0i	IO	2	
pfrelr	CORNER		4

=====

Timing/Optimization Information:

Global Routing Information:

layer METAL, dir Hor, min width = 0.16, min space = 0.18 pitch = 0.41
layer METAL2, dir Ver, min width = 0.20, min space = 0.21 pitch = 0.41
layer METAL3, dir Hor, min width = 0.20, min space = 0.21 pitch = 0.41
layer METAL4, dir Ver, min width = 0.20, min space = 0.21 pitch = 0.51
layer METAL5, dir Hor, min width = 0.40, min space = 0.40 pitch = 0.81
layer METAL6, dir Ver, min width = 0.44, min space = 0.46 pitch = 0.97

Average gCell capacity	1.26	on layer (1)	METAL
Average gCell capacity	1.58	on layer (2)	METAL2
Average gCell capacity	1.21	on layer (3)	METAL3
Average gCell capacity	1.00	on layer (4)	METAL4
Average gCell capacity	4.39	on layer (5)	METAL5
Average gCell capacity	3.68	on layer (6)	METAL6

Initial. Both Dirs: Overflow =	6	Max = 1	GRCs =	6	(0.00%)
Initial. H routing: Overflow =	1	Max = 1	(GRCs = 1)	GRCs =	1 (0.00%)
Initial. V routing: Overflow =	5	Max = 1	(GRCs = 5)	GRCs =	5 (0.00%)

phase1. Both Dirs: Overflow =	6	Max = 1	GRCs =	6	(0.00%)
-------------------------------	---	---------	--------	---	---------

phase1. H routing: Overflow = 1 Max = 1 (GRCs = 1) GRCs = 1 (0.00%)
phase1. V routing: Overflow = 5 Max = 1 (GRCs = 5) GRCs = 5 (0.00%)

phase2. Both Dirs: Overflow = 6 Max = 1 GRCs = 6 (0.00%)
phase2. H routing: Overflow = 1 Max = 1 (GRCs = 1) GRCs = 1 (0.00%)
phase2. V routing: Overflow = 5 Max = 1 (GRCs = 5) GRCs = 5 (0.00%)

Total Wire Length = 266.89
Layer METAL wire length = 0.00
Layer METAL2 wire length = 153.76
Layer METAL3 wire length = 113.12
Layer METAL4 wire length = 0.00
Layer METAL5 wire length = 0.00
Layer METAL6 wire length = 0.00
Total Number of Contacts = 85
Via VIA12A count = 47
Via VIA23 count = 38
Via VIA34 count = 0
Via VIA45 count = 0
Via VIA56 count = 0

Elapsed real time: 0:00:00
Elapsed cpu time: sys = 0:00:00 usr = 0:00:00 total = 0:00:00
Total Proc Memory(MB): 519

Track Assignment Information:

Number of wires with overlap after iteration 0 = 136 of 242

Number of wires with overlap after iteration 1 = 91 of 201

Total metal1 wire length: 21.1
Total metal2 wire length: 199.3
Total metal3 wire length: 133.4
Total metal4 wire length: 5.5
Total metal5 wire length: 6.6
Total metal6 wire length: 0.0
Total wire length: 365.9

Elapsed real time: 0:00:00
Elapsed cpu time: sys = 0:00:00 usr = 0:00:00 total = 0:00:00
Total Proc Memory(MB): 523

Detailed Routing Information:

----- Detail route -----

DR finished with 0 violations

DRC-SUMMARY:

@@@@@@@ TOTAL VIOLATIONS = 0

----- Chip finish: insert redundant vias -----

RedundantVia finished with 0 violations

DRC-SUMMARY:

@@@@@@@ TOTAL VIOLATIONS = 0

Begin DRC fixing ...

Elapsed real time: 0:00:03

Elapsed cpu time: sys = 0:00:00 usr = 0:00:01 total = 0:00:01

Total Proc Memory(MB): 470

Cumulative run time upto current stage: Elapsed = 0:00:03 CPU = 0:00:01

DR finished with 0 open nets, of which 0 are frozen

DR finished with 0 violations

DRC-SUMMARY:

@@@@@@@ TOTAL VIOLATIONS = 0

Total number of nets = 1360

0 open nets, of which 0 are frozen

Total number of excluded ports = 0 ports of 0 unplaced cells connected to 0 nets

0 ports without pins of 0 cells connected to 0 nets

0 ports of 0 cover cells connected to 0 non-pg nets

Total number of DRCs = 0

Total number of antenna violations = antenna checking not active

Total number of voltage-area violations = no voltage-areas defined

Elapsed real time: 0:00:17
Elapsed cpu time: sys = 0:00:00 usr = 0:00:00 total = 0:00:00
Total Proc Memory(MB): 470

Cumulative run time upto current stage: Elapsed = 0:00:20 CPU = 0:00:01

----- Chip finish: insert redundant vias -----

RedundantVia finished with 0 violations

DRC-SUMMARY:

@@@@@@@ TOTAL VIOLATIONS = 0

Begin DRC fixing ...

Elapsed real time: 0:00:01
Elapsed cpu time: sys = 0:00:00 usr = 0:00:01 total = 0:00:01
Total Proc Memory(MB): 510

Cumulative run time upto current stage: Elapsed = 0:00:21 CPU = 0:00:02

Redundant via insertion finished with 0 open nets, of which 0 are frozen
Redundant via insertion finished with 0 violations

DRC-SUMMARY:

@@@@@@@ TOTAL VIOLATIONS = 0

Total number of nets = 1360

0 open nets, of which 0 are frozen

Total number of excluded ports = 0 ports of 0 unplaced cells connected to 0 nets

0 ports without pins of 0 cells connected to 0 nets

0 ports of 0 cover cells connected to 0 non-pg nets

Total number of DRCs = 0

Total number of antenna violations = antenna checking not active

Total number of voltage-area violations = no voltage-areas defined

Elapsed real time: 0:00:00

Elapsed cpu time: sys = 0:00:00 usr = 0:00:00 total = 0:00:00

Total Proc Memory(MB): 510

Cumulative run time upto current stage: Elapsed = 0:00:21 CPU = 0:00:02

Total Wire Length = 90759 micron

Total Number of Contacts = 10836

Total Number of Wires = 15220

Total Number of PtConns = 2388

Total Number of Routable Wires = 15220

Total Routable Wire Length = 90759 micron

Layer METAL : 9499 micron

Layer METAL2 : 32049 micron

Layer METAL3 : 40791 micron

Layer METAL4 : 4598 micron

Layer METAL5 : 2279 micron

Layer METAL6 : 1542 micron

Via VIA56_2x1 : 107

Via VIA45_1x2 : 93

Via VIA45_2x1 : 7

Via VIA45(rot)_1x2 : 5

Via VIA34 : 6

Via VIA34(rot) : 3

Via VIA34f : 2

Via VIA34(rot)_2x1 : 12

Via VIA34_2x1 : 448

Via VIA34(rot)_1x2 : 2

Via VIA34_1x2 : 4

Via VIA23 : 141

Via VIA23f : 4

Via VIA23_1x2 : 3925

Via VIA23_2x1 : 733

Via VIA23(rot)_2x1 : 3

Via VIA23(rot)_1x2 : 1

Via VIA12A : 528

Via VIA12A(rot) : 1

Via VIA12B : 102

Via VIA12f : 6

Via VIA12A_2x1 : 1881

Via VIA12A_1x2 : 482

Via	VIA12B_1x2 :	1007
Via	VIA12B(rot)_2x1 :	530
Via	VIA12A(rot)_1x2 :	681
Via	VIA12A(rot)_2x1 :	76
Via	VIA12B_2x1 :	37
Via	VIA12B(rot)_1x2 :	9

Redundant via conversion report:

Total optimized via conversion rate = 92.79% (10055 / 10836 vias)

Layer VIA	= 88.18%	(4709 / 5340 vias)
Weight 1	= 88.18%	(4709 vias)
Un-optimized	= 11.82%	(631 vias)
Layer VIA2	= 97.07%	(4666 / 4807 vias)
Weight 1	= 97.07%	(4666 vias)
Un-optimized	= 2.93%	(141 vias)
Layer VIA3	= 98.11%	(468 / 477 vias)
Weight 1	= 98.11%	(468 vias)
Un-optimized	= 1.89%	(9 vias)
Layer VIA4	= 100.00%	(105 / 105 vias)
Weight 1	= 100.00%	(105 vias)
Un-optimized	= 0.00%	(0 vias)
Layer VIA5	= 100.00%	(107 / 107 vias)
Weight 1	= 100.00%	(107 vias)
Un-optimized	= 0.00%	(0 vias)

Total double via conversion rate = 92.68% (10043 / 10836 vias)

Layer VIA	= 88.07%	(4703 / 5340 vias)
Layer VIA2	= 96.98%	(4662 / 4807 vias)
Layer VIA3	= 97.69%	(466 / 477 vias)
Layer VIA4	= 100.00%	(105 / 105 vias)
Layer VIA5	= 100.00%	(107 / 107 vias)

DRC information:

Antenna: 54

Total error number: 54

Ring Wiring Statistics:

metal2 Wire Length(count):	2.80(126)
metal3 Wire Length(count):	3370.40(258)
metal4 Wire Length(count):	2245.80(382)

=====

Total Wire Length(count):	5619.00(766)
Number of via3 Contacts:	12

=====

Total Number of Contacts:	12
---------------------------	----

Stripe Wiring Statistics:

metal4 Wire Length(count):	1118.70(2)
----------------------------	------------

=====

Total Wire Length(count):	1118.70(2)
Number of via3 Contacts:	6

=====

Total Number of Contacts:	6
---------------------------	---

User Wiring Statistics:

PG follow-pin Wiring Statistics:

metal1 Wire Length(count):	47647.40(85)
----------------------------	--------------

=====

Total Wire Length(count):	47647.40(85)
Number of via1 Contacts:	268
Number of via2 Contacts:	268
Number of via3 Contacts:	249

=====

Total Number of Contacts:	785
---------------------------	-----

Signal Wiring Statistics:

metal1 Wire Length(count):	9616.69(2015)
metal2 Wire Length(count):	32463.49(8974)
metal3 Wire Length(count):	40817.10(6279)
metal4 Wire Length(count):	4599.40(285)
metal5 Wire Length(count):	2279.88(112)
metal6 Wire Length(count):	1547.90(101)

=====

Total Wire Length(count):	91324.46(17766)
---------------------------	-----------------

Mask Name	Contact Code	Number Of Contacts	Percentage
-----------	--------------	--------------------	------------

via1	VIA12A(1)	529	9.91
via1	VIA12B(2)	102	1.91
via1	VIA12f(9)	6	0.112
via1_2x1	VIA12B(2)	46	0.861
via1_1x2	VIA12B(2)	1537	28.8
via1_1x2	VIA12A(1)	558	10.4
via1_2x1	VIA12A(1)	2562	48
Default via for layer via1:		11.9%	
Yield-optimized via for layer via1:		88.1%	

via2	VIA23(3)	141	2.93
via2	VIA23f(10)	4	0.0832
via2_2x1	VIA23(3)	734	15.3
via2_1x2	VIA23(3)	3928	81.7
Default via for layer via2:		3.02%	
Yield-optimized via for layer via2:		97%	

via3	VIA34(4)	9	1.89
via3	VIA34f(11)	2	0.419
via3_2x1	VIA34(4)	450	94.3
via3_1x2	VIA34(4)	16	3.35
Default via for layer via3:		2.31%	
Yield-optimized via for layer via3:		97.7%	

via4_2x1	VIA45(5)	12	11.4
via4_1x2	VIA45(5)	93	88.6
Default via for layer via4:		0%	
Yield-optimized via for layer via4:		100%	

via5_2x1	VIA56(6)	107	100
Default via for layer via5:		0%	
Yield-optimized via for layer via5:		100%	

Double Via rate for all layers: 92.7%

=====

Total Number of Contacts: 10836

Horizontal/Vertical Wire Distribution:

Layer	Hor. Wire (% of Hor.)	Ver. Wire (% of Ver.)
metal1	9190.34 (17.47%)	426.35 (1.10%)

metal2	375.58 (0.71%)	32087.90 (82.88%)
metal3	40741.72 (77.45%)	75.37 (0.19%)
metal4	9.62 (0.02%)	4589.78 (11.85%)
metal5	2275.36 (4.33%)	4.52 (0.01%)
metal6	13.54 (0.03%)	1534.36 (3.96%)
=====		
Total	52606.18	38718.28

LVS Run Time:

Elapsed = 0:00:00, CPU = 0:00:00

1

3-Quality Of Results Report

Report : qor

Design : RISC_CHIP

Version: E-2010.12-ICC-SP2

Date : Thu Nov 8 11:07:21 2012

Timing Path Group 'INPUTS'

Levels of Logic: 2.00
Critical Path Length: 1.15
Critical Path Slack: 1.52
Critical Path Clk Period: 8.00
Total Negative Slack: 0.00
No. of Violating Paths: 0.00
Worst Hold Violation: 0.00
Total Hold Violation: 0.00
No. of Hold Violations: 0.00

Timing Path Group 'OUTPUTS'

Levels of Logic: 4.00
Critical Path Length: 5.05
Critical Path Slack: 1.05
Critical Path Clk Period: 8.00
Total Negative Slack: 0.00
No. of Violating Paths: 0.00
Worst Hold Violation: 0.00
Total Hold Violation: 0.00
No. of Hold Violations: 0.00

Timing Path Group 'clk'

Levels of Logic: 24.00
Critical Path Length: 6.22
Critical Path Slack: 1.46

Critical Path Clk Period: 8.00
Total Negative Slack: 0.00
No. of Violating Paths: 0.00
Worst Hold Violation: 0.00
Total Hold Violation: 0.00
No. of Hold Violations: 0.00

Cell Count

Hierarchical Cell Count: 22
Hierarchical Port Count: 1135
Leaf Cell Count: 1210
Buf/Inv Cell Count: 135
CT Buf/Inv Cell Count: 6
Combinational Cell Count: 963
Sequential Cell Count: 247
Macro Count: 0

Area

Combinational Area: 111628.664062
Noncombinational Area: 11193.860352
Net Area: 1370.016051
Net XLength : 53884.93
Net YLength : 39485.69

Cell Area: 122822.524414
Design Area: 124192.540465
Net Length : 93370.61

Design Rules

Total Number of Nets: 1343
Nets With Violations: 0

Hostname: eesun04.ecs.csun.edu

Compile CPU Statistics

```
-----  
Resource Sharing:          0.00  
Logic Optimization:       0.00  
Mapping Optimization:     2.93  
-----  
Overall Compile Time:      3.32  
Overall Compile Wall Clock Time: 3.76
```

1

4-Power Report

Report : power

-analysis_effort low

Design : RISC_CHIP

Version: E-2010.12-ICC-SP2

Date : Thu Nov 8 11:08:34 2012

Library(s) Used:

cb13fs120_tsmc_max (File: /home/users5/ma581955/ref/db/sc_max.db)

cb13io320_tsmc_max (File: /home/users5/ma581955/ref/db/io_max.db)

ram16x128_max (File: /home/users5/ma581955/ref/db/ram16x128_max.db)

Operating Conditions: cb13fs120_tsmc_max Library: cb13fs120_tsmc_max
Wire Load Model Mode: enclosed

Design	Wire Load Model	Library

RISC_CHIP	140000	cb13fs120_tsmc_max
REG_FILE	16000	cb13fs120_tsmc_max
INSTRN_LAT	8000	cb13fs120_tsmc_max
ALU	8000	cb13fs120_tsmc_max
PRGRM_FSM	ForQA	cb13fs120_tsmc_max
ALU_DW01_add_1	ForQA	cb13fs120_tsmc_max
ALU_DW01_add_0	ForQA	cb13fs120_tsmc_max
ALU_DW01_inc_0	ForQA	cb13fs120_tsmc_max
ALU_DW01_sub_1	ForQA	cb13fs120_tsmc_max
ALU_DW01_sub_0	ForQA	cb13fs120_tsmc_max
ALU_DW01_dec_0	ForQA	cb13fs120_tsmc_max
PRGRM_CNT_DW01_inc_0	ForQA	cb13fs120_tsmc_max

Global Operating Voltage = 1.08

Power-specific unit information :

Voltage Units = 1V

Capacitance Units = 1.000000pf

Time Units = 1ns

Dynamic Power Units = 1mW (derived from V,C,T units)
Leakage Power Units = 1pW

Cell Internal Power = 49.3431 mW (95%)
Net Switching Power = 2.3906 mW (5%)

Total Dynamic Power = 51.7338 mW (100%)

Cell Leakage Power = 89.1487 uW

1